

UCUENCA

INGENIERÍA

CARRERA DE TELECOMUNICACIONES

PERÍODO 202664-PA01 · MAR 2026 – AGO 2026

Manual de Prácticas

Redes Inalámbricas

INGE-00100 · Nivel 8 · 144 horas

Ing. Fabián Astudillo-Salinas, PhD.

Cuenca – Ecuador 2026

Índice general

1	Presentación del Manual	5
1.1	Estructura de las prácticas	5
1.2	Carga horaria	5
1.3	Resultados de aprendizaje	6
1.4	Normas generales de laboratorio	6
1.5	Mapa de prácticas del curso	6
2	P-C2: Sistema de Control de Acceso con RFID	8
2.1	Marco Teórico	8
2.1.1	Identificación por Radiofrecuencia (RFID)	8
2.1.2	Bandas de frecuencia y estándares	8
2.1.3	RFID vs. NFC	9
2.1.4	El módulo RFID-RC522 (MFRC522)	9
2.1.5	La plataforma NodeMCU ESP32	9
2.1.6	Seguridad en sistemas RFID	9
2.2	Objetivos	10
2.3	Materiales y Equipos	10
2.4	Descripción del Proyecto	11
2.5	Diagrama de Conexión	12
2.5.1	ESP32 ↔ RC522 (bus SPI)	12
2.5.2	LED RGB y buzzer	12
2.6	Procedimiento	12
2.6.1	Paso 1 — Configuración del entorno de desarrollo	12
2.6.2	Paso 2 — Montaje del circuito	13
2.6.3	Paso 3 — Lectura básica del UID	13
2.6.4	Paso 4 — Implementación del servidor de acceso	14
2.6.5	Paso 5 — Código principal del sistema	15
2.6.6	Paso 6 — Pruebas del sistema	17
2.7	Preguntas de Análisis	17
2.8	Entregables	18
2.9	Rúbrica de Evaluación	18
3	P-C3: Control de Acceso con Bluetooth y App Android	20
3.1	Marco Teórico	20
3.1.1	Bluetooth Classic y el perfil SPP	20
3.1.2	Proceso de emparejamiento Bluetooth	21
3.1.3	Bluetooth en Android	21
3.1.4	Seguridad en sistemas de acceso Bluetooth	22
3.2	Objetivos	22
3.3	Materiales y Equipos	23
3.4	Descripción del sistema	23

3.5	Procedimiento	24
3.5.1	Paso 1 — Configuración del entorno de desarrollo	24
3.5.2	Paso 2 — Montaje del circuito	24
3.5.3	Paso 3 — Implementación del servidor de acceso	25
3.5.4	Paso 4 — Firmware del ESP32 (servidor SPP)	25
3.5.5	Paso 5 — Desarrollo de la aplicación Android	29
3.5.6	Paso 6 — Pruebas del sistema	30
3.6	Preguntas de Análisis	31
3.7	Entregables	31
3.8	Rúbrica de Evaluación	32
4	P-C4: Análisis de Tráfico Wi-Fi y Configuración de AP	34
4.1	Marco Teórico	34
4.1.1	El estándar IEEE 802.11 (Wi-Fi)	34
4.1.2	Arquitectura WLAN	34
4.1.3	Trama IEEE 802.11	34
4.1.4	Seguridad WLAN	35
4.2	Objetivos	35
4.3	Materiales y Equipos	35
4.4	Procedimiento	36
4.4.1	Paso 1 — Escaneo pasivo del espectro	36
4.4.2	Paso 2 — Análisis de tramas Beacon	36
4.4.3	Paso 3 — Configuración del AP	36
4.4.4	Paso 4 — Medición de rendimiento con iPerf3	37
4.4.5	Paso 5 — Captura del handshake WPA2	37
4.5	Preguntas de Análisis	37
4.6	Entregables	37
4.7	Rúbrica de Evaluación	38
5	P-C5: Comunicación Bluetooth Low Energy entre ESP32	39
5.1	Marco Teórico	39
5.1.1	Bluetooth y el estándar IEEE 802.15	39
5.1.2	Bluetooth Low Energy (BLE)	39
5.1.3	BLE en el ESP32	39
5.2	Objetivos	40
5.3	Materiales y Equipos	40
5.4	Procedimiento	40
5.4.1	Paso 0 — Configuración del entorno de desarrollo	41
5.4.2	Paso 1 — ESP32 Peripheral (servidor BLE)	41
5.4.3	Paso 2 — ESP32 Central (cliente BLE)	42
5.4.4	Paso 3 — Medición de alcance	43
5.5	Preguntas de Análisis	43
5.6	Entregables	44
5.7	Rúbrica de Evaluación	44

6	P-C6: Red de Sensores con ZigBee / IEEE 802.15.4	45
6.1	Marco Teórico	45
6.1.1	El estándar IEEE 802.15.4	45
6.1.2	ZigBee sobre IEEE 802.15.4	45
6.1.3	Roles de dispositivos ZigBee	46
6.1.4	Módulos XBee	46
6.1.5	Estructura de la trama IEEE 802.15.4	46
6.2	Objetivos	47
6.3	Materiales y Equipos	47
6.4	Procedimiento	47
6.4.1	Paso 1 — Instalación y configuración con XCTU	48
6.4.2	Paso 2 — Comunicación punto a punto (transparente)	48
6.4.3	Paso 3 — Lectura del sensor DHT22 con ESP32 (End Device)	49
6.4.4	Paso 4 — Recepción en el Coordinador	50
6.4.5	Paso 5 — Topología en malla	50
6.5	Preguntas de Análisis	50
6.6	Entregables	51
6.7	Rúbrica de Evaluación	51
7	P-C7: Enlace LoRa entre Gateway y Mote	52
7.1	Marco Teórico	52
7.1.1	LoRa y LoRaWAN	52
7.1.2	Parámetros de enlace LoRa	52
7.1.3	Ecuación de enlace de Friis	53
7.1.4	Arquitectura de red LoRaWAN	53
7.2	Objetivos	53
7.3	Materiales y Equipos	54
7.4	Procedimiento	54
7.4.1	Paso 0 — Configuración del entorno de desarrollo	54
7.4.2	Paso 1 — Configuración del Mote (transmisor)	55
7.4.3	Paso 2 — Configuración del Gateway (receptor)	56
7.4.4	Paso 3 — Medición de enlace vs. Spreading Factor	56
7.4.5	Paso 4 — Presupuesto de enlace	57
7.5	Preguntas de Análisis	57
7.6	Entregables	57
7.7	Rúbrica de Evaluación	58
8	P-C8: Auditoría de Seguridad en Redes Wi-Fi	59
8.1	Marco Teórico	59
8.1.1	Amenazas en redes inalámbricas	59
8.1.2	Herramientas de auditoría	59
8.2	Objetivos	60
8.3	Materiales y Equipos	60
8.4	Procedimiento	61
8.4.1	Paso 1 — Preparación del entorno Kali	61
8.4.2	Paso 2 — Escaneo de redes	61
8.4.3	Paso 3 — Ataque de diccionario	61

8.4.4	Paso 4 — Plan de hardening	62
8.5	Preguntas de Análisis	62
8.6	Entregables	62
8.7	Rúbrica de Evaluación	63
9	P-C9: Diseño de Red Inalámbrica Empresarial	64
9.1	Marco Teórico	64
9.1.1	Proceso de diseño de redes inalámbricas	64
9.1.2	Herramientas de diseño	64
9.2	Objetivos	65
9.3	Escenario	65
9.4	Procedimiento	66
9.4.1	Paso 1 — Cálculo de capacidad	66
9.4.2	Paso 2 — Diseño en Cisco Packet Tracer	66
9.4.3	Paso 3 — Verificación	66
9.5	Preguntas de Análisis	66
9.6	Entregables	67
9.7	Rúbrica de Evaluación	67
	Referencias Bibliográficas	68

Capítulo 1

Presentación del Manual

Este *Manual de Prácticas* es el documento guía de la componente **Aprendizaje Práctico Experimental (APE)** de la asignatura **Redes Inalámbricas** (INGE-00100), Nivel 8 de la Carrera de Telecomunicaciones de la Universidad de Cuenca.

Cada práctica sigue la estructura que se indica en la Tabla 1.1 y está alineada con una o varias unidades del sílabo oficial.

1.1 Estructura de las prácticas

Cuadro 1.1: Estructura de cada práctica

Sección	Contenido
Marco teórico	Fundamentos para contextualizar la práctica.
Objetivos	Competencias que se desarrollan (RdA alineados al sílabo).
Materiales	Hardware, software y otros recursos requeridos.
Procedimiento	Pasos detallados de montaje, configuración y programación.
Análisis	Preguntas de reflexión y experimentación adicional.
Entregables	Documentos o artefactos que el grupo debe entregar.
Rúbrica	Criterios y ponderaciones de evaluación.

1.2 Carga horaria

Cuadro 1.2: Componentes de aprendizaje — Redes Inalámbricas (INGE-00100)

Componente	Horas
Aprendizaje Contacto Docente (ACD)	48
Aprendizaje Práctico Experimental (APE)	48
Aprendizaje Autónomo (AA)	48
Total	144

1.3 Resultados de aprendizaje

- RdA1.** Describir las características de las diferentes tecnologías de redes inalámbricas, incluyendo arquitectura, protocolos y seguridad.
- RdA2.** Identificar y analizar las diferentes aplicaciones de las redes inalámbricas.
- RdA3.** Diseñar y configurar redes inalámbricas seguras y efectivas para satisfacer las necesidades específicas de una organización.
- RdA4.** Aplicar herramientas y tecnologías relevantes para las redes inalámbricas en proyectos prácticos para afianzar el conocimiento adquirido y compartir experiencias.

1.4 Normas generales de laboratorio

- Presentarse puntualmente con los materiales indicados en cada práctica.
- Seguir las instrucciones de seguridad para el manejo de equipos electrónicos: verificar polaridades antes de energizar circuitos.
- Entregar un **único informe por grupo** (máximo 3 integrantes).
- Citar correctamente todas las fuentes consultadas (formato IEEE).
- El plagio implica calificación de cero y reporte disciplinario.

1.5 Mapa de prácticas del curso

Cuadro 1.3: Prácticas programadas por unidad temática del sílabo

Cód.	Unidad	Práctica	Tecnología
P-C2	200. Comunicaciones de campo cercano	Sistema de control de acceso con RFID	RC522 + ESP32
P-C3	200. Comunicaciones de campo cercano	Control de acceso con Bluetooth y app Android	BT Classic + ESP32
P-C4	300. WLAN / IEEE 802.11	Análisis de tráfico Wi-Fi y configuración AP	Wireshark
P-C5	400. Bluetooth / IEEE 802.15	Comunicación BLE entre dos ESP32	Bluetooth LE

P-C6	500. ZigBee / IEEE 802.15.4	Red de sensores ZigBee	XBee
P-C7	600. LoRa / Cobertura amplia	Enlace LoRa: Gate- way y Mote	LoRa / TTN
P-C8	700. Seguridad	Auditoría de segu- ridad Wi-Fi	Kali / Aircrack-ng
P-C9	800–900. Diseño empresarial	Diseño de red in- alámbrica empre- sarial	Packet Tracer

Capítulo 2

P-C2: Sistema de Control de Acceso con RFID

Unidad:	200 – Campo cercano (NFC/RFID)	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

2.1 Marco Teórico

2.1.1 Identificación por Radiofrecuencia (RFID)

La **Identificación por Radiofrecuencia** (RFID, *Radio Frequency Identification*) es una tecnología de comunicación inalámbrica que permite la identificación automática de objetos, personas o animales mediante campos electromagnéticos de corto alcance.

Un sistema RFID consta de tres componentes fundamentales:

1. **Etiqueta (Tag):** circuito integrado con antena que almacena un identificador único (UID) y, opcionalmente, datos adicionales. Las etiquetas *pasivas* no tienen batería: se energizan por inducción magnética del campo del lector. Las *activas* incorporan batería propia y pueden operar a mayor distancia.
2. **Lector (Reader/Interrogator):** genera el campo electromagnético, energiza la etiqueta y lee o escribe datos.
3. **Sistema de back-end:** base de datos y software de gestión que procesa los datos leídos y controla la lógica de la aplicación.

2.1.2 Bandas de frecuencia y estándares

Cuadro 2.1: Principales bandas de operación RFID

Banda	Frecuencia	Alcance	Aplicaciones típicas
LF	125–134 kHz	<10 cm	Control de animales, acceso vehicular
HF	13,56 MHz	<1 m	Tarjetas de acceso, pasaportes, NFC
UHF	860–960 MHz	1–12 m	Logística, inventarios, peajes
Microondas	2,45/5,8 GHz	>10 m	Peajes de alta velocidad

El módulo **MFRC522** opera a **13,56 MHz** (banda HF) e implementa el estándar ISO/IEC 14443-A, compatible con tarjetas Mifare Classic y llaveros NFC.

2.1.3 RFID vs. NFC

i RFID y NFC: diferencias clave

NFC (*Near Field Communication*) es un subconjunto del ecosistema RFID que opera a 13,56 MHz con un alcance máximo de ≈ 20 cm. La diferencia principal es la *bidireccionalidad*: NFC permite comunicación activo-activo (dos dispositivos se leen mutuamente), mientras que RFID clásico es unidireccional (lector $\rightarrow$ etiqueta pasiva). NFC sigue los estándares ISO 18092 y ECMA 340.

2.1.4 El módulo RFID-RC522 (MFRC522)

Cuadro 2.2: Especificaciones del módulo MFRC522

Parámetro	Valor
Frecuencia de operación	13,56 MHz
Distancia de lectura	hasta 5 cm (tarjeta ISO 14443 estándar)
Interfaz	SPI (hasta 10 Mbps) / I ² C / UART
Tensión de alimentación	2,5–3,3 V
Corriente de operación	13–26 mA
Estándares soportados	ISO/IEC 14443-A/B, Mifare

A Voltaje de alimentación

El módulo RC522 opera a **3,3 V**. Conectarlo a 5 V puede dañarlo irreversiblemente. La placa ESP32 provee 3,3 V en el pin 3V3.

2.1.5 La plataforma NodeMCU ESP32

El **ESP32** (Espressif) es un SoC con procesador Xtensa LX6 de doble núcleo a 240 MHz, Wi-Fi 802,11 b/g/n, Bluetooth 4,2/BLE, 34 GPIOs programables y múltiples interfaces (SPI, I²C, UART, ADC, DAC). Su conectividad Wi-Fi lo hace ideal para proyectos IoT que requieren enviar datos a un servidor remoto.

2.1.6 Seguridad en sistemas RFID

Los sistemas RFID presentan vulnerabilidades específicas:

- **Clonación de UID:** con hardware de bajo costo se puede copiar el identificador de una tarjeta pasiva.
- **Eavesdropping:** interceptación pasiva de la comunicación RF.
- **Relay attacks:** extensión artificial del alcance de lectura mediante dos dispositivos radio retransmisores.
- **Brute-force en sectores Mifare:** el cifrado propietario CRYPTO1 de Mifare Classic ha sido comprometido (*Darkside attack*, 2008).

Contramedidas recomendadas: cifrado AES-128 (Mifare DESFire), autenticación mutua, registro de eventos, alertas por intentos fallidos repetidos y validación server-side (nunca confiar únicamente en el UID).

2.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Implementar un sistema funcional de control de acceso utilizando la tecnología RFID junto con la plataforma ESP32, integrando hardware, firmware y un servidor remoto de gestión de credenciales.

Objetivos específicos

- OE1.** Describir el principio de funcionamiento de RFID y sus características (arquitectura, protocolo SPI, estándar ISO 14443).
- OE2.** Diseñar e implementar el circuito de conexión RC522 + ESP32 + LED RGB + buzzer.
- OE3.** Programar la lógica de control de acceso que verifique credenciales en una base de datos remota y registre cada evento.
- OE4.** Evaluar la seguridad del sistema e identificar posibles vulnerabilidades con propuestas de mejora.

2.3 Materiales y Equipos

Materiales y Equipos

Cant.	Componente	Referencia	Observaciones
1	Placa NodeMCU ESP32	ESP32-WROOM-32	30 o 38 pines
1	Módulo lector RFID	MFRC522 / RC522	Incluye tarjeta y llavero
2	Tag RFID	Mifare Classic 1K	13,56 MHz
1	LED RGB cátodo común	—	O tres LEDs discretos
1	Buzzer activo 5 V	—	Señal DC (sin oscilador externo)
3	Resistencia 220 Ω	—	Para los tres canales del LED
1	Protoboard	—	830 puntos mínimo
—	Cables Dupont	M-M / M-H	Varios colores
1	Cable USB tipo-C	—	Programación ESP32
1	Computador	—	VSCoDe + extensión Espressif IDF

Software: VSCoDe, extensión **Espressif IDF**, ESP-IDF ≥ 5.1 , comp. `abobija/esp-rc522`, `cJSON` (built-in ESP-IDF), servidor HTTP (Flask/Node.js).

2.4 Descripción del Proyecto

El proyecto consiste en diseñar e implementar un **sistema de control de acceso** para una instalación específica. El sistema deberá:

1. Leer el UID de una tarjeta o llavero RFID al acercarlo al módulo RC522.
2. Consultar una **base de datos centralizada en un servidor externo** para verificar si el UID está autorizado. Basarse en el proyecto <https://github.com/Yasouimo/IOT-door-lock>.
3. Indicar el resultado mediante **LED RGB**:
 -  Verde: acceso concedido.
 -  Rojo: acceso denegado.
 -  Azul: sistema listo / en espera.
4. Emitir retroalimentación sonora con el **buzzer**: pitido corto para acceso concedido; pitido largo para acceso denegado.
5. **Registrar** en la base de datos la fecha, hora, UID y resultado de cada intento de acceso.

Requisito arquitectónico obligatorio

La base de datos **no puede residir en el ESP32**. Se requiere comunicación entre el ESP32 y un servidor de gestión de acceso mediante protocolo HTTP (REST API) o MQTT.

2.5 Diagrama de Conexión

2.5.1 ESP32 ↔ RC522 (bus SPI)

Cuadro 2.3: Conexión módulo RC522 → ESP32

Pin RC522	Función SPI	Pin ESP32	Color cable sugerido
SDA (SS/CS)	Chip Select	GPIO 5	Amarillo
SCK	Reloj SPI	GPIO 18	Naranja
MOSI	Master Out Slave In	GPIO 23	Verde
MISO	Master In Slave Out	GPIO 19	Azul
IRQ	Interrupción (opcional)	—	—
GND	Tierra	GND	Negro
RST	Reset	GPIO 22	Blanco
3,3 V	Alimentación	3V3	Rojo

2.5.2 LED RGB y buzzer

Cuadro 2.4: Conexión LED RGB (cátodo común) y buzzer

Componente	Terminal	Resistencia	GPIO ESP32
LED RGB – Rojo	Ánodo R	220 Ω	GPIO 25
LED RGB – Verde	Ánodo G	220 Ω	GPIO 26
LED RGB – Azul	Ánodo B	220 Ω	GPIO 27
LED RGB – Cátodo	Común	—	GND
Buzzer activo (+)	Pin +	—	GPIO 32
Buzzer activo (—)	Pin —	—	GND

i LED RGB de ánodo común

Si su LED RGB es de **ánodo común**, conecte el pin común a 3,3 V e invierta la lógica en el código: LOW enciende, HIGH apaga.

2.6 Procedimiento

2.6.1 Paso 1 — Configuración del entorno de desarrollo

1. Instale **Visual Studio Code** desde <https://code.visualstudio.com>.
2. En VSCode abra el panel de extensiones (**Ctrl+Shift+X**), busque **Espressif IDF** (ID: `espressif.esp-idf-extension`) e instálela.

3. Configure el SDK: abra la Paleta de Comandos (`Ctrl+Shift+P`) y ejecute *ESP-IDF: Configure ESP-IDF Extension*. Seleccione **Express**, elija **ESP-IDF v5.x** y deje que el asistente descargue el SDK y la cadena de herramientas Xtensa-GCC (≈500 MB, solo la primera vez).
4. Cree el proyecto: *ESP-IDF: New Project* → nombre `rfid_acceso`, chip ESP32.
5. Declare las dependencias en `main/idf_component.yml`:

```

1 dependencies:
2   idf: ">=5.1"
3   abobija/esp-rc522: "^2.0.0"

```

Listing 2.1: `main/idf_component.yml` — P-C2 RFID

cJSON está incluido en ESP-IDF y no requiere declaración adicional. Ejecute `idf.py update-dependencies` para descargar los componentes.

6. Verifique que `main/CMakeLists.txt` liste el fuente:

```

1 idf_component_register(SRCS "main.c"
2                        INCLUDE_DIRS ".")

```

Listing 2.2: `main/CMakeLists.txt` — P-C2

7. Compile: *ESP-IDF: Build your project* (`Ctrl+E, B`) o `idf.py build` en terminal.
8. Seleccione el puerto (*ESP-IDF: Select Port to Use*), luego cargue: *ESP-IDF: Flash your project* (`Ctrl+E, F`).
9. Monitor serie: *ESP-IDF: Monitor your device* (`Ctrl+E, M`) o `idf.py monitor` — 115 200 baud.

2.6.2 Paso 2 — Montaje del circuito

1. Monte el ESP32 en la protoboard dejando un rail de alimentación libre a cada lado.
2. Conecte el RC522 según la Tabla 2.3. **Verifique que la alimentación sea 3,3 V.**
3. Conecte el LED RGB con resistencias según la Tabla 2.4.
4. Conecte el buzzer activo al GPIO 32.
5. Antes de energizar, verifique todas las conexiones con multímetro en modo continuidad.

2.6.3 Paso 3 — Lectura básica del UID

Cargue el siguiente programa para confirmar que el módulo RC522 funciona correctamente y anotar los UIDs de sus tarjetas de prueba. El código lo podemos bajar del repositorio github <https://github.com/fabianastudillo/RI-practicas>.

```

1 #include "esp_log.h"
2 #include "esp_rc522.h"
3
4 #define PIN_MISO 19
5 #define PIN_MOSI 23

```

```

6 #define PIN_SCLK 18
7 #define PIN_CS 5
8
9 static const char *TAG = "RFID";
10 static esp_rc522_handle_t scanner;
11
12 static void on_tag_scanned(void *arg, esp_event_base_t base,
13                             int32_t id, void *data)
14 {
15     esp_rc522_event_data_t *ev = (esp_rc522_event_data_t *)data;
16     if (id == RC522_EVENT_TAG_SCANNED) {
17         esp_rc522_tag_t *tag = (esp_rc522_tag_t *)ev->ptr;
18         ESP_LOGI(TAG, "UID: " RC522_UID_FMT,
19                 RC522_UID_ARG(tag->serial_number));
20     }
21 }
22
23 void app_main(void)
24 {
25     esp_rc522_spi_config_t spi = {
26         .host = SPI2_HOST,
27         .miso_gpio = PIN_MISO,
28         .mosi_gpio = PIN_MOSI,
29         .sck_gpio = PIN_SCLK,
30         .sda_gpio = PIN_CS,
31     };
32     esp_rc522_config_t cfg = { .transport =
33                               ESP_RC522_TRANSPORT_SPI,
34                               .transport_config.spi = &spi };
35     ESP_ERROR_CHECK(esp_rc522_create(&cfg, &scanner));
36     ESP_ERROR_CHECK(esp_rc522_register_events(scanner,
37         RC522_EVENT_ANY, on_tag_scanned, NULL));
38     ESP_ERROR_CHECK(esp_rc522_start(scanner));
39     ESP_LOGI(TAG, "Acerque una tarjeta RFID...");
40 }

```

Listing 2.3: Lectura del UID con esp-rc522 — ESP-IDF

Abra el *Monitor Serie* a **115200 bps** y registre los UIDs.

2.6.4 Paso 4 — Implementación del servidor de acceso

Implemente un servidor HTTP sencillo (Python/Flask o Node.js/Express) que exponga la siguiente API REST:

Cuadro 2.5: API REST del servidor de control de acceso

Método	Endpoint	Descripción
--------	----------	-------------

GET	/check?uid={UID}	Verifica autorización.	Retorna: {"autorizado":true/false,"nombre":"..."}
POST	/log	Registra evento.	Body: {"uid","timestamp","resultado"}
GET	/usuarios	Lista todos los usuarios registrados.	
POST	/usuarios	Registra un nuevo usuario con su UID.	

2.6.5 Paso 5 — Código principal del sistema

```

1 #include <string.h>
2 #include "esp_log.h"
3 #include "esp_wifi.h"
4 #include "esp_event.h"
5 #include "esp_netif.h"
6 #include "esp_http_client.h"
7 #include "esp_rc522.h"
8 #include "cJSON.h"
9 #include "driver/gpio.h"
10 #include "freertos/FreeRTOS.h"
11 #include "freertos/task.h"
12
13 #define SSID "TU_SSID"
14 #define PASSWORD "TU_CONTRASENA"
15 #define SERVER "http://192.168.1.100:5000"
16 #define PIN_CS 5
17 #define LED_R 25
18 #define LED_G 26
19 #define LED_B 27
20 #define BUZZER 32
21
22 static const char *TAG = "RFID_ACC";
23 static esp_rc522_handle_t scanner;
24 static char uid_str[20];
25
26 // ---- LED y Buzzer ----
27 static void set_led(bool r, bool g, bool b) {
28     gpio_set_level(LED_R, r);
29     gpio_set_level(LED_G, g);
30     gpio_set_level(LED_B, b);
31 }
32 static void beep(uint32_t ms) {
33     gpio_set_level(BUZZER, 1);
34     vTaskDelay(pdMS_TO_TICKS(ms));
35     gpio_set_level(BUZZER, 0);
36 }
37
38 // ---- Verificar acceso via HTTP ----
39 static bool verificar_acceso(const char *uid) {

```



```

40     char url[128];
41     snprintf(url, sizeof(url), "%s/check?uid=%s", SERVER, uid);
42     char buf[256] = {0};
43     esp_http_client_config_t cfg = { .url = url,
44         .user_data = buf, .buffer_size_tx = 256 };
45     esp_http_client_handle_t c = esp_http_client_init(&cfg);
46     esp_err_t err = esp_http_client_perform(c);
47     esp_http_client_cleanup(c);
48     if (err != ESP_OK) return false;
49     cJSON *root = cJSON_Parse(buf);
50     bool ok = cJSON_IsTrue(cJSON_GetObjectItem(root,
51         "autorizado"));
52     cJSON_Delete(root);
53     return ok;
54 }
55 // ---- Callback RFID ----
56 static void on_tag_scanned(void *arg, esp_event_base_t base,
57     int32_t id, void *data) {
58     if (id != RC522_EVENT_TAG_SCANNED) return;
59     esp_rc522_tag_t *tag =
60         ((esp_rc522_event_data_t *)data)->ptr;
61     snprintf(uid_str, sizeof(uid_str),
62         RC522_UID_FMT, RC522_UID_ARG(tag->serial_number));
63     ESP_LOGI(TAG, "Tarjeta: %s", uid_str);
64     set_led(1, 1, 1); // Blanco: procesando
65     bool ok = verificar_acceso(uid_str);
66     if (ok) {
67         ESP_LOGI(TAG, "ACCESO CONCEDIDO");
68         set_led(0, 1, 0); beep(150);
69     } else {
70         ESP_LOGI(TAG, "ACCESO DENEGADO");
71         set_led(1, 0, 0); beep(600);
72     }
73     vTaskDelay(pdMS_TO_TICKS(2500));
74     set_led(0, 0, 1); // Azul: en espera
75 }
76
77 void app_main(void) {
78     // GPIOs
79     gpio_config_t io = { .pin_bit_mask =
80         (1ULL<<LED_R)|(1ULL<<LED_G)|(1ULL<<LED_B)|(1ULL<<BUZZER),
81         .mode = GPIO_MODE_OUTPUT };
82     gpio_config(&io);
83     set_led(1, 1, 0); // Amarillo: conectando
84
85     // Wi-Fi
86     esp_netif_init();
87     esp_event_loop_create_default();
88     esp_netif_create_default_wifi_sta();

```

```
89  wifi_init_config_t wcfg = WIFI_INIT_CONFIG_DEFAULT();
90  esp_wifi_init(&wcfg);
91  esp_wifi_set_mode(WIFI_MODE_STA);
92  wifi_config_t wsta = { .sta = { .ssid = SSID,
93                                .password = PASSWORD } };
94  esp_wifi_set_config(WIFI_IF_STA, &wsta);
95  esp_wifi_start();
96  esp_wifi_connect();
97  vTaskDelay(pdMS_TO_TICKS(3000));    // Esperar conexion
98  ESP_LOGI(TAG, "Wi-Fi conectado");
99  set_led(0, 0, 1);                  // Azul: listo
100
101  // RFID
102  esp_rc522_spi_config_t spi = { .host = SPI2_HOST,
103                                .miso_gpio=19, .mosi_gpio=23, .sck_gpio=18,
104                                .sda_gpio=PIN_CS };
105  esp_rc522_config_t rcfg = { .transport =
106                              ESP_RC522_TRANSPORT_SPI,
107                              .transport_config.spi = &spi };
108  ESP_ERROR_CHECK(esp_rc522_create(&rcfg, &scanner));
109  ESP_ERROR_CHECK(esp_rc522_register_events(scanner,
110                                             RC522_EVENT_ANY, on_tag_scanned, NULL));
111  ESP_ERROR_CHECK(esp_rc522_start(scanner));
112  ESP_LOGI(TAG, "Sistema listo. Acerque su tarjeta.");
113 }
```

Listing 2.4: Sistema completo de control de acceso RFID — ESP-IDF

2.6.6 Paso 6 — Pruebas del sistema

1. Registre al menos **dos UUIDs** en la base de datos: uno autorizado y uno no autorizado.
2. Pruebe ambos casos y verifique la respuesta visual (LED) y sonora (buzzer).
3. Consulte la ruta `/log` y confirme que los eventos quedan registrados con fecha, hora y UID.
4. **Prueba de fallo de red:** desconecte el Wi-Fi o detenga el servidor y documente el comportamiento del sistema.
5. **Prueba de escalabilidad:** agregue cinco usuarios y realice diez lecturas consecutivas midiendo el tiempo de respuesta.

2.7 Preguntas de Análisis

Redacte un **ensayo técnico** (mínimo 800 palabras) que responda las siguientes preguntas. El ensayo debe incluir referencias bibliográficas en formato IEEE y estar integrado en el informe de práctica.

- P1.** ¿Qué es RFID y cuál es su principio de funcionamiento? Explique el proceso de transferencia de energía inductiva en etiquetas pasivas.
- P2.** ¿Cuáles son las características principales de RFID (frecuencias, estándares, tipos de etiqueta, rango de lectura)?
- P3.** Explique la diferencia entre RFID y NFC: protocolos involucrados, casos de uso y limitaciones de cada tecnología.
- P4.** Mencione al menos **cinco** aplicaciones industriales o comerciales de RFID distintas al control de acceso de personas.
- P5.** ¿Qué vulnerabilidades de seguridad presenta el sistema implementado? Proponga al menos dos contramedidas concretas.
- P6.** ¿Por qué es importante que la base de datos resida en un servidor externo y no en el ESP32? Analice escalabilidad, disponibilidad y seguridad.

2.8 Entregables

Entregables

1. **Informe técnico en PDF** (un archivo por grupo):
 - Portada: asignatura, práctica, integrantes del grupo, fecha.
 - Marco teórico: ensayo respondiendo las preguntas de análisis.
 - Diseño del sistema: diagrama de bloques y tabla de conexiones.
 - Código fuente comentado (completo, sin omisiones).
 - Capturas: Monitor Serie, tabla de eventos del servidor, pruebas.
 - Análisis de resultados y conclusiones.
 - Bibliografía en formato IEEE (≥ 5 fuentes).
2. **Demostración en vivo** en sesión de laboratorio: el sistema debe funcionar con al menos una tarjeta autorizada y una no autorizada, mostrando respuesta visual y sonora.
3. **Repositorio de código** (GitHub o GitLab): enlace incluido en la última página del informe.

2.9 Rúbrica de Evaluación

Cuadro 2.6: Rúbrica de evaluación — Práctica P-C2 (10 puntos)

Criterio	Peso	Indicador	Descripción
Presentación y defensa	20 %	Exposición oral	Vocalización, ritmo y confianza adecuados; responde preguntas del docente; se ajusta al tiempo asignado (10 min).
Informe escrito	20 %	Estructura y formato	Portada, índice, secciones diferenciadas, paginación correcta.
		Contenido técnico	Resume los aspectos clave; cita todas las fuentes; no incluye material superfluo.
		Figuras, tablas y código	Imágenes legibles; elementos numerados y referenciados en el texto.
		Redacción y ortografía	Redacción técnica correcta; frases claras y concisas.
Calidad técnica	60 %	Cumplimiento de hitos	Sistema funcional: lectura RFID, verificación remota, LED y buzzer operativos.
		Análisis del problema	Diseño del sistema justificado; subproblemas identificados y resueltos.
		Integración servidor-ESP32	Comunicación HTTP implementada; registro de eventos funcional con fecha y hora.
		Seguridad	Identifica ≥ 2 vulnerabilidades y propone contramedidas viables.
		Síntesis y creatividad	Solución original; considera mejoras al enunciado base.

Cuadro 2.7: Escala de calificación

Nivel	Porcentaje alcanzado	Puntos (sobre 10)
Excelente	90–100 %	9,0–10,0
Muy bueno	75–89 %	7,5–8,9
Suficiente	60–74 %	6,0–7,4
Insuficiente	<60 %	<6,0

Capítulo 3

P-C3: Control de Acceso con Bluetooth y App Android

Unidad:	200 – Campo cercano / Bluetooth	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

3.1 Marco Teórico

3.1.1 Bluetooth Classic y el perfil SPP

Bluetooth Classic (IEEE 802.15.1) es un estándar de comunicación inalámbrica de corto alcance que opera en la banda ISM de 2,4 GHz. A diferencia de BLE (*Bluetooth Low Energy*), Bluetooth Classic está diseñado para transferencias continuas de datos con mayor ancho de banda.

El **Serial Port Profile (SPP)** emula un enlace serie RS-232 sobre Bluetooth, proporcionando un canal bidireccional de datos entre dos dispositivos. Es el perfil más utilizado en aplicaciones IoT que requieren comunicación interactiva, ya que:

- Permite transmisión en ambos sentidos (full-duplex).
- No requiere definir servicios GATT complejos (a diferencia de BLE).
- Es compatible de forma nativa con la API Bluetooth de Android.
- Soporta autenticación y cifrado mediante emparejamiento.

Cuadro 3.1: Comparación entre Bluetooth Classic y BLE

Característica	BT Classic	BLE	Observación
Velocidad máx.	3 Mbps (EDR)	2 Mbps (5.0)	Classic ofrece mayor throughput sostenido
Consumo	Medio-alto	Ultra bajo	BLE diseñado para baterías de botón
Alcance típico	10–100 m	10–200 m	Depende de la potencia de transmisión
Emparejamiento	PIN / SSP	Just Works / Passkey	Classic requiere emparejamiento explícito

Perfil para datos	SPP, A2DP, HFP	GATT	SPP es ideal para flujos serie
API Android	BluetoothSocket	BluetoothGatt	SPP usa RFCOMM

3.1.2 Proceso de emparejamiento Bluetooth

El emparejamiento (*pairing*) establece una relación de confianza entre dos dispositivos mediante el intercambio de claves criptográficas. El flujo general es:

1. **Descubrimiento:** el dispositivo *Central* (Android) escanea dispositivos cercanos; el *Peripheral* (ESP32) se anuncia como *discoverable*.
2. **Solicitud de emparejamiento:** el Central envía una solicitud; el usuario confirma en ambos dispositivos (o acepta un PIN).
3. **Intercambio de claves:** se genera una *link key* compartida mediante *Secure Simple Pairing* (SSP) o ingreso de PIN.
4. **Cifrado:** las comunicaciones posteriores se cifran con la clave compartida (AES-CCM en BT 4.0+).
5. **Conexión SPP:** una vez emparejados, el Central puede abrir un socket RFCOMM al servicio SPP del Peripheral.

i Nota

Emparejamiento vs. conexión: El emparejamiento se realiza una sola vez; las claves se almacenan en NVS (*Non-Volatile Storage*) del ESP32 y en el sistema Android. Las reconexiones posteriores son automáticas y cifradas sin intervención del usuario.

3.1.3 Bluetooth en Android

Android proporciona una API Bluetooth robusta en el paquete `android.bluetooth`. Los componentes clave para esta práctica son:

- **BluetoothAdapter:** representa el adaptador BT local; permite descubrir dispositivos y gestionar emparejamientos.
- **BluetoothDevice:** representa un dispositivo remoto; se obtiene por descubrimiento o de la lista de emparejados.
- **BluetoothSocket:** socket RFCOMM para comunicación SPP; se crea con `device.createRfcommSocket`.
- **InputStream / OutputStream:** flujos de lectura/escritura sobre el socket, similares a un puerto serie.

⚠ Advertencia

A partir de Android 12 (API 31) es necesario solicitar los permisos `BLUETOOTH_CONNECT` y `BLUETOOTH_SCAN` en tiempo de ejecución. En versiones anteriores basta con `BLUETOOTH` y `BLUETOOTH_ADMIN` en el manifiesto.

3.1.4 Seguridad en sistemas de acceso Bluetooth

Los sistemas de control de acceso basados en Bluetooth presentan vulnerabilidades específicas que deben considerarse:

1. **Eavesdropping:** interceptación de la comunicación si el cifrado no está activo o es débil.
2. **Replay attacks:** un atacante captura un token válido y lo retransmite posteriormente.
3. **Relay attacks:** extensión artificial del alcance retransmitiendo paquetes entre dos dispositivos atacantes.
4. **Suplantación de dispositivo:** clonación de la dirección MAC Bluetooth del ESP32 para crear un punto de acceso falso.
5. **Fuerza bruta del PIN:** si se usa un PIN numérico corto, puede ser adivinado por enumeración.

Contramedidas recomendadas:

- Usar SSP (*Secure Simple Pairing*) con MITM protection.
- Implementar tokens temporales (*one-time tokens* o TOTP) en lugar de credenciales estáticas.
- Validar siempre en el servidor: el ESP32 no debe tomar decisiones de acceso por sí solo.
- Registrar todos los intentos con marca temporal para auditoría.
- Limitar reintentos fallidos (lockout temporal).

3.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Implementar un sistema de control de acceso basado en Bluetooth Classic, en el que una aplicación Android se empareje con un ESP32 y envíe credenciales de autenticación verificadas contra un servidor remoto.

Objetivos específicos

- OE1.** Describir los principios de operación de Bluetooth Classic, el perfil SPP y el proceso de emparejamiento seguro.
- OE2.** Diseñar e implementar el circuito ESP32 + LED RGB + buzzer para retroalimentación visual y auditiva del acceso.
- OE3.** Programar el firmware del ESP32 como servidor SPP que reciba credenciales desde Android y las verifique contra un servidor remoto vía HTTP.

- OE4.** Desarrollar una aplicación Android que descubra el ESP32, se empareje y envíe el token de autenticación vía SPP.
- OE5.** Evaluar la seguridad del sistema, identificar vulnerabilidades y proponer mejoras concretas.

3.3 Materiales y Equipos

✂ Materiales y Equipos

Cant.	Componente	Ref.	Observaciones
1	Placa NodeMCU ESP32	ESP32-WROOM-32	30 o 38 pines
1	LED RGB	Cátodo común	O 3 LEDs discretos
1	Buzzer activo	5V	Señal DC
3	Resistencia	220 Ω	Canales R, G, B
1	Protoboard	—	830 puntos mínimo
—	Cables Dupont	M-M / M-H	Varios colores
1	Cable USB tipo-C	—	Programación ESP32
1	Computador	—	VSCode + Espressif IDF
1	Teléfono Android	API $\geq$ 21	Con Bluetooth activado

Software: VSCode, extensión **Espressif IDF**, ESP-IDF $\geq$ 5.1, cJSON (built-in), servidor HTTP (Flask/Node.js), Android Studio (Kotlin o Java).

3.4 Descripción del sistema

El sistema de control de acceso con Bluetooth debe cumplir los siguientes requisitos funcionales:

1. El ESP32 se anuncia como dispositivo Bluetooth Classic con nombre **ESP32-Acceso** y acepta conexiones SPP.
2. El usuario abre la app Android, se empareja con el ESP32 (primera vez) y envía un token de autenticación.
3. El ESP32 recibe el token vía SPP y lo verifica consultando un servidor HTTP remoto.
4. El ESP32 indica el resultado mediante LED RGB: **verde** (acceso concedido), **rojo** (acceso denegado), **azul** (sistema en espera).

5. El buzzer emite un tono corto (150 ms) para acceso concedido o largo (600 ms) para acceso denegado.
6. El ESP32 envía la respuesta (**GRANTED** o **DENIED**) de vuelta a la app Android por SPP.
7. Cada intento de acceso se registra en la base de datos del servidor con fecha, hora, token y resultado.

i Nota

Requisito arquitectónico: Al igual que en la práctica RFID (P-C2), la base de datos de usuarios debe residir en un servidor externo, no en el ESP32. La comunicación entre el ESP32 y el servidor se realiza mediante HTTP REST API, y el ESP32 se conecta al servidor vía Wi-Fi.

3.5 Procedimiento

3.5.1 Paso 1 — Configuración del entorno de desarrollo

Siga los pasos 1–4 de P-C2 (VSCode + extensión Espressif IDF + ESP-IDF v5.x). Cree el proyecto con nombre `bt_acceso` y chip ESP32.

En `main/idf_component.yml` declare las dependencias:

```
1 dependencies:
2   idf: ">=5.1"
```

Listing 3.1: `main/idf_component.yml` — P-C3 Bluetooth

⚠ Advertencia

Bluetooth Classic y Wi-Fi coexisten en el ESP32 pero comparten la radio de 2,4 GHz. En `menuconfig` (*Component config* → *Bluetooth*) active:

- Bluetooth → Enabled
- Bluetooth controller mode → BR/EDR Only
- Bluedroid → Classic Bluetooth → SPP

Verifique que `main/CMakeLists.txt` contenga:

```
1 idf_component_register(SRCS "main.c"
2                       INCLUDE_DIRS ".")
```

Listing 3.2: `main/CMakeLists.txt` — P-C3

3.5.2 Paso 2 — Montaje del circuito

El circuito es idéntico al de la práctica P-C2 (RFID), **sin** el módulo RC522. Se utilizan los mismos GPIOs para LED RGB y buzzer:

Cuadro 3.2: Conexiones LED RGB y buzzer — P-C3

Componente	Terminal	Resistencia	GPIO
LED Rojo	Ánodo R	220 Ω	GPIO 25
LED Verde	Ánodo G	220 Ω	GPIO 26
LED Azul	Ánodo B	220 Ω	GPIO 27
LED	Cátodo	—	GND
Buzzer (+)	Positivo	—	GPIO 32
Buzzer (-)	Negativo	—	GND

Monte el ESP32 en la protoboard, conecte el LED RGB con las resistencias de protección y el buzzer. Verifique todas las conexiones con un multímetro antes de alimentar.

3.5.3 Paso 3 — Implementación del servidor de acceso

Implemente un servidor HTTP idéntico al de P-C2, adaptando el endpoint de verificación para recibir un **token** en lugar de un UID:

Cuadro 3.3: API REST del servidor — P-C3

Método	Endpoint	Descripción
GET	/check?token={TOKEN}	Verifica autorización. Retorna: {"autorizado":true/false,"nombre":"..."}
POST	/log	Registra evento. Body: {"token","timestamp","resultado"}
GET	/usuarios	Lista todos los usuarios registrados.
POST	/usuarios	Registra un nuevo usuario con su token.

Cada usuario registrado tiene un **token** único (cadena alfanumérica) que se almacena en la base de datos del servidor y se configura en la app Android del usuario.

3.5.4 Paso 4 — Firmware del ESP32 (servidor SPP)

Cargue el código del archivo `code/bt_access.c`, que implementa el sistema completo de control de acceso por Bluetooth. El código se puede descargar del repositorio <https://github.com/fabianastudillo/RI-practicas>.

```

1 #include <string.h>
2 #include "esp_log.h"
3 #include "esp_bt.h"
4 #include "esp_bt_main.h"
5 #include "esp_bt_device.h"

```

```

6 #include "esp_gap_bt_api.h"
7 #include "esp_spp_api.h"
8 #include "esp_wifi.h"
9 #include "esp_event.h"
10 #include "esp_netif.h"
11 #include "esp_http_client.h"
12 #include "cJSON.h"
13 #include "driver/gpio.h"
14 #include "freertos/FreeRTOS.h"
15 #include "freertos/task.h"
16 #include "nvs_flash.h"
17
18 #define SSID          "TU_SSID"
19 #define PASSWORD      "TU_CONTRASENA"
20 #define SERVER        "http://192.168.1.100:5000"
21 #define LED_R         25
22 #define LED_G         26
23 #define LED_B         27
24 #define BUZZER        32
25 #define DEVICE_NAME   "ESP32-Acceso"
26
27 static const char *TAG = "BT_ACC";
28
29 // ---- LED y Buzzer ----
30 static void set_led(bool r, bool g, bool b) {
31     gpio_set_level(LED_R, r);
32     gpio_set_level(LED_G, g);
33     gpio_set_level(LED_B, b);
34 }
35 static void beep(uint32_t ms) {
36     gpio_set_level(BUZZER, 1);
37     vTaskDelay(pdMS_TO_TICKS(ms));
38     gpio_set_level(BUZZER, 0);
39 }
40
41 // ---- Verificar acceso via HTTP ----
42 static bool verificar_acceso(const char *token) {
43     char url[128];
44     snprintf(url, sizeof(url), "%s/check?token=%s", SERVER, token);
45     char buf[256] = {0};
46     esp_http_client_config_t cfg = { .url = url,
47                                     .user_data = buf, .buffer_size_tx = 256 };
48     esp_http_client_handle_t c = esp_http_client_init(&cfg);
49     esp_err_t err = esp_http_client_perform(c);
50     esp_http_client_cleanup(c);
51     if (err != ESP_OK) return false;
52     cJSON *root = cJSON_Parse(buf);
53     bool ok = cJSON_IsTrue(cJSON_GetObjectItem(root,
54         "autorizado"));
54     cJSON_Delete(root);

```

```

55     return ok;
56 }
57
58 // ---- Callback SPP ----
59 static void spp_callback(esp_spp_cb_event_t event,
60                          esp_spp_cb_param_t *param) {
61     switch (event) {
62     case ESP_SPP_INIT_EVT:
63         esp_bt_dev_set_device_name(DEVICE_NAME);
64         esp_bt_gap_set_scan_mode(ESP_BT_CONNECTABLE,
65                                   ESP_BT_GENERAL_DISCOVERABLE);
66         esp_spp_start_srv(ESP_SPP_SEC_AUTHENTICATE,
67                           ESP_SPP_ROLE_SLAVE, 0, "SPP_SERVER");
68         break;
69     case ESP_SPP_START_EVT:
70         ESP_LOGI(TAG, "Servidor SPP iniciado");
71         break;
72     case ESP_SPP_SRV_OPEN_EVT:
73         ESP_LOGI(TAG, "Cliente conectado");
74         set_led(1, 1, 1); // Blanco: procesando
75         break;
76     case ESP_SPP_DATA_IND_EVT: {
77         char token[64] = {0};
78         int len = param->data_ind.len < 63 ? param->data_ind.len :
79             63;
80         memcpy(token, param->data_ind.data, len);
81         token[len] = '\0';
82         while (len > 0 && (token[len-1]=='\n' ||
83                           token[len-1]=='\r'))
84             token[--len] = '\0';
85         ESP_LOGI(TAG, "Token recibido: %s", token);
86
87         bool ok = verificar_acceso(token);
88         const char *resp;
89         if (ok) {
90             ESP_LOGI(TAG, "ACCESO CONCEDIDO");
91             set_led(0, 1, 0); beep(150);
92             resp = "GRANTED\n";
93         } else {
94             ESP_LOGI(TAG, "ACCESO DENEGADO");
95             set_led(1, 0, 0); beep(600);
96             resp = "DENIED\n";
97         }
98         esp_spp_write(param->data_ind.handle,
99                       strlen(resp), (uint8_t *)resp);
100        vTaskDelay(pdMS_TO_TICKS(2500));
101        set_led(0, 0, 1); // Azul: en espera
102        break;
103    }
104     case ESP_SPP_CLOSE_EVT:

```

```

103     ESP_LOGI(TAG, "Cliente desconectado");
104     set_led(0, 0, 1);
105     break;
106 default:
107     break;
108 }
109 }
110
111 void app_main(void) {
112     // GPIOs
113     gpio_config_t io = { .pin_bit_mask =
114         (1ULL<<LED_R)|(1ULL<<LED_G)|(1ULL<<LED_B)|(1ULL<<BUZZER),
115         .mode = GPIO_MODE_OUTPUT };
116     gpio_config(&io);
117     set_led(1, 1, 0); // Amarillo:
118                         inicializando
119
120     // NVS (requerido por BT y Wi-Fi)
121     nvs_flash_init();
122
123     // Wi-Fi
124     esp_netif_init();
125     esp_event_loop_create_default();
126     esp_netif_create_default_wifi_sta();
127     wifi_init_config_t wcfg = WIFI_INIT_CONFIG_DEFAULT();
128     esp_wifi_init(&wcfg);
129     esp_wifi_set_mode(WIFI_MODE_STA);
130     wifi_config_t wsta = { .sta = { .ssid = SSID,
131                                     .password = PASSWORD } };
132     esp_wifi_set_config(WIFI_IF_STA, &wsta);
133     esp_wifi_start();
134     esp_wifi_connect();
135     vTaskDelay(pdMS_TO_TICKS(3000));
136     ESP_LOGI(TAG, "Wi-Fi conectado");
137
138     // Bluetooth Classic
139     esp_bt_controller_config_t bt_cfg =
140         BT_CONTROLLER_INIT_CONFIG_DEFAULT();
141     esp_bt_controller_init(&bt_cfg);
142     esp_bt_controller_enable(ESP_BT_MODE_CLASSIC_BT);
143     esp_bluedroid_init();
144     esp_bluedroid_enable();
145     esp_spp_register_callback(spp_callback);
146     esp_spp_init(ESP_SPP_MODE_CB);
147
148     set_led(0, 0, 1); // Azul: listo
149     ESP_LOGI(TAG, "Sistema listo. Conecte desde la app Android.");
150 }

```

Listing 3.3: Sistema de control de acceso Bluetooth — ESP-IDF

Flujo del firmware:

1. Inicializa GPIOs (LED amarillo: *inicializando*).
2. Inicializa NVS (requerido por Bluetooth y Wi-Fi).
3. Conecta a Wi-Fi (para comunicarse con el servidor HTTP).
4. Inicializa Bluetooth Classic con pila Bluedroid.
5. Registra callback SPP e inicia servidor SPP.
6. LED azul: *sistema listo*.
7. Al recibir datos SPP: extrae token, consulta servidor HTTP, activa LED/buzzer según resultado y envía respuesta al cliente.

Configure `TU_SSID`, `TU_CONTRASENA` y la IP del servidor en las macros del código antes de compilar.

3.5.5 Paso 5 — Desarrollo de la aplicación Android

Desarrolle una aplicación Android con las siguientes funcionalidades:

1. Pantalla principal:

- Botón “Buscar dispositivos” que inicia el descubrimiento Bluetooth.
- Lista de dispositivos encontrados (nombre + dirección MAC).
- Campo de texto para ingresar el token de acceso del usuario.
- Botón “Solicitar acceso” que envía el token al ESP32.
- Área de estado que muestra la respuesta recibida (**GRANTED** / **DENIED**).

2. Flujo de conexión:

- a) Verificar que Bluetooth está activado; si no, solicitar activación.
- b) Escanear dispositivos y filtrar por nombre **ESP32-Acceso**.
- c) Emparejar (si es la primera vez) y conectar vía **BluetoothSocket** con UUID SPP: **00001101-0000-1000-8000-00805F9B34FB**.
- d) Enviar el token por el **OutputStream** del socket.
- e) Leer la respuesta por el **InputStream**.
- f) Mostrar el resultado al usuario y cerrar el socket.

```

1 // UUID estandar para SPP
2 val SPP_UUID = UUID.fromString(
3     "00001101-0000-1000-8000-00805F9B34FB")
4
5 // Obtener dispositivo emparejado
6 val device: BluetoothDevice =
7     adapter.bondedDevices.first { it.name == "ESP32-Acceso" }
8
9 // Conectar

```

```
10 val socket = device
11     .createRfcommSocketToServiceRecord(SPP_UUID)
12 socket.connect()
13
14 // Enviar token
15 val out = socket.getOutputStream
16 out.write("MI_TOKEN_SECRETO\n".toByteArray())
17 out.flush()
18
19 // Leer respuesta
20 val inp = socket.getInputStream
21 val buffer = ByteArray(64)
22 val bytes = inp.read(buffer)
23 val respuesta = String(buffer, 0, bytes).trim()
24 // respuesta = "GRANTED" o "DENIED"
25
26 socket.close()
```

Listing 3.4: Fragmento de conexión SPP en Android (Kotlin)

Advertencia

Las operaciones Bluetooth en Android son bloqueantes y **no deben ejecutarse en el hilo principal** (UI thread). Utilice `CoroutineScope(Dispatchers.IO)` en Kotlin o un `AsyncTask/Thread` en Java.

Permisos requeridos en `AndroidManifest.xml`:

```
1 <uses-permission
2     android:name="android.permission.BLUETOOTH" />
3 <uses-permission
4     android:name="android.permission.BLUETOOTH_ADMIN" />
5 <uses-permission
6     android:name="android.permission.BLUETOOTH_CONNECT" />
7 <uses-permission
8     android:name="android.permission.BLUETOOTH_SCAN" />
9 <uses-permission
10    android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Listing 3.5: Permisos Bluetooth en `AndroidManifest.xml`

3.5.6 Paso 6 — Pruebas del sistema

1. Registre al menos 2 tokens en el servidor (uno autorizado y uno no autorizado).
2. Desde la app Android, conecte al ESP32 y envíe el token autorizado. Verifique: LED verde, beep corto, respuesta **GRANTED** en la app.
3. Envíe el token no autorizado. Verifique: LED rojo, beep largo, respuesta **DENIED**.
4. Consulte el endpoint `/log` del servidor y confirme que ambos eventos quedaron registrados con marca temporal.

5. **Prueba de alcance:** repita el acceso a 1, 5, 10 y 15 m de distancia. Registre si la conexión SPP se establece correctamente.
6. **Prueba de fallo de red:** desconecte el servidor o la Wi-Fi del ESP32 y observe el comportamiento (el acceso debe denegarse).
7. **Prueba de reconexión:** apague y encienda Bluetooth en el teléfono; verifique que el emparejamiento persiste y no solicita nuevo PIN.

3.6 Preguntas de Análisis

Redacte un ensayo técnico (mínimo 800 palabras, con citas IEEE) que responda las siguientes preguntas:

- P1.** Explique las diferencias entre Bluetooth Classic y BLE en cuanto a arquitectura, consumo, velocidad y casos de uso típicos.
- P2.** Describa el proceso de emparejamiento *Secure Simple Pairing* (SSP): fases, intercambio de claves y protección contra ataques MITM.
- P3.** Compare este sistema de acceso (Bluetooth + Android) con el sistema RFID de la práctica P-C2. ¿Cuáles son las ventajas y desventajas de cada enfoque?
- P4.** Identifique al menos 3 vulnerabilidades de seguridad del sistema implementado y proponga contramedidas concretas para cada una.
- P5.** ¿Por qué es necesario que la verificación de credenciales se realice en un servidor remoto y no localmente en el ESP32? Analice desde las perspectivas de escalabilidad, seguridad y mantenimiento.
- P6.** Proponga una mejora al sistema que utilice tokens temporales (por ejemplo, TOTP) en lugar de tokens estáticos. Describa el flujo modificado.

3.7 Entregables

Entregables

1. Informe técnico (PDF):

- Portada con datos del curso, práctica, integrantes y fecha.
- Marco teórico: ensayo respondiendo las preguntas de análisis.
- Diseño del sistema: diagramas de bloques y tabla de conexiones.
- Código fuente completo y comentado (ESP32 y Android).
- Capturas de pantalla: Monitor Serie, app Android, log del servidor.
- Análisis de resultados y conclusiones.
- Bibliografía en formato IEEE (≥ 5 fuentes).

2. **Demostración en vivo:** el sistema debe funcionar con al menos 1 token autorizado y 1 no autorizado, mostrando respuesta visual, auditiva y en la app Android.
3. **Repositorio de código:** enlace a GitHub/GitLab con el código del ESP32 y el proyecto Android completo.

3.8 Rúbrica de Evaluación

Cuadro 3.4: Rúbrica de evaluación — Práctica P-C3 (10 puntos)

Criterio	Peso	Indicador	Descripción
Presentación y defensa	20 %	Exposición oral	Vocalización, ritmo, seguridad; responde preguntas; respeta límite de 10 min.
		Demostración en vivo	Sistema funciona con token autorizado y denegado; app Android muestra resultado.
Informe escrito	20 %	Estructura/formato	Portada, índice, secciones, paginación correctos.
		Contenido	Aspectos clave resumidos; fuentes citadas; sin material superfluo.
		Figuras/tablas/código	Imágenes claras; elementos numerados y referenciados.
		Redacción y ortografía	Lenguaje técnico correcto; oraciones claras y concisas.
Calidad técnica	60 %	Hitos de implementación	Funcional: emparejamiento BT, envío de token, verificación remota, LED y buzzer operativos.
		App Android	Aplicación funcional: descubrimiento, emparejamiento, envío de token y recepción de respuesta.
		Integración servidor-ESP32	Comunicación HTTP implementada; registro de eventos funcional con marca temporal.
		Seguridad	Identifica ≥ 3 vulnerabilidades y propone contramedidas viables.

Síntesis y creatividad Solución original; considera mejoras al enunciado base (p. ej. TOTP, interfaz mejorada).

Cuadro 3.5: Escala de calificación — P-C3

Nivel	Porcentaje alcanzado	Puntos (sobre 10)
Excelente	90–100 %	9,0–10,0
Muy bueno	75–89 %	7,5–8,9
Suficiente	60–74 %	6,0–7,4
Insuficiente	<60 %	<6,0

Capítulo 4

P-C4: Análisis de Tráfico Wi-Fi y Configuración de AP

Unidad:	300 – WLAN / IEEE 802.11	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

4.1 Marco Teórico

4.1.1 El estándar IEEE 802.11 (Wi-Fi)

El estándar **IEEE 802.11** define las especificaciones para redes de área local inalámbricas (WLAN). Las enmiendas más relevantes en uso actual son:

Cuadro 4.1: Principales enmiendas del estándar IEEE 802.11

Estándar	Año	Banda	Velocidad máx.	Características clave
802.11n	2009	2,4/5 GHz	600 Mbps	MIMO, canales de 40 MHz
802.11ac	2013	5 GHz	3,5 Gbps	MU-MIMO, beamforming, canales hasta 160 MHz
802.11ax	2021	2,4/5/6 GHz	9,6 Gbps	Wi-Fi 6/6E, OFDMA, BSS Coloring

4.1.2 Arquitectura WLAN

Una WLAN puede operar en dos modos fundamentales:

- **Modo infraestructura:** los clientes se asocian a un punto de acceso (AP) que actúa como concentrador. Un AP y sus clientes forman un *Basic Service Set* (BSS), identificado por un BSSID (dirección MAC del AP). El nombre de la red visible al usuario es el *SSID*.
- **Modo ad hoc (IBSS):** los dispositivos se comunican directamente entre sí, sin necesidad de AP.

4.1.3 Trama IEEE 802.11

La trama de gestión *Beacon* que emite periódicamente el AP contiene: SSID, BSSID, capacidades, tasas soportadas, parámetros de seguridad (RSN/WPA) e información de temporización. Wireshark permite capturar y diseccionar estas tramas para auditar redes cercanas.

4.1.4 Seguridad WLAN

Cuadro 4.2: Evolución de los mecanismos de seguridad Wi-Fi

Protocolo	Año	Estado
WEP	1997	Obsoleto – roto en minutos con herramientas como aircrack-ng
WPA	2003	Deprecado – TKIP vulnerable a ataques de reinstalación
WPA2	2004	Vigente – AES-CCMP; vulnerable a KRACK (parcheable)
WPA3	2018	Recomendado – SAE (Dragonfly), PMF obligatorio

4.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Analizar el tráfico de una red WLAN mediante Wireshark y configurar un punto de acceso seguro para comprender los mecanismos del estándar IEEE 802.11.

Objetivos específicos

- OE1.** Capturar e identificar tramas de gestión 802.11 (Beacon, Probe Request/Response, Authentication, Association).
- OE2.** Comparar las métricas de rendimiento (throughput, latencia, RSSI) en distintas bandas y canales.
- OE3.** Configurar un AP con cifrado WPA2-AES y verificar el proceso de autenticación en Wireshark.
- OE4.** Identificar redes con cifrado débil o nulo en el entorno y proponer mejoras de seguridad.

4.3 Materiales y Equipos

Materiales y Equipos

Cant.	Componente	Ref.	Observaciones
1	Router/AP inalámbrico	TP-Link o similar	Acceso a consola web
2	Computadores portátiles	—	Con adaptador Wi-Fi
1	Adaptador Wi-Fi USB	—	Modo monitor (p.ej. Alfa AWUS036)
1	Software Wireshark	≥ 4.0	https://www.wireshark.org
1	Software iPerf3	—	Medición de throughput

4.4 Procedimiento

4.4.1 Paso 1 — Escaneo pasivo del espectro

1. En Linux, habilite el modo monitor en el adaptador Wi-Fi:

```

1 sudo ip link set wlan0 down
2 sudo iw wlan0 set monitor control
3 sudo ip link set wlan0 up

```

Listing 4.1: Activar modo monitor

2. Abra Wireshark, seleccione la interfaz en modo monitor y aplique el filtro: `wlan.fc.type_subtype == 0x08` para capturar únicamente tramas *Beacon*.
3. Capture durante 60 segundos. Exporte el resultado en formato `.pcapng`.

4.4.2 Paso 2 — Análisis de tramas Beacon

Para cada red detectada registre en una tabla:

Cuadro 4.3: Plantilla de registro de redes detectadas

SSID	BSSID	Canal	Banda	Seguridad	Observaciones
—	—	—	—	—	—

4.4.3 Paso 3 — Configuración del AP

1. Acceda a la consola web del AP (`192.168.0.1`).
2. Configure el SSID, canal, banda y cifrado **WPA2-AES**. Documente cada parámetro seleccionado.
3. Conecte ambos portátiles a la red configurada y verifique la asignación de IPs.

4.4.4 Paso 4 — Medición de rendimiento con iPerf3

1. En el PC servidor: `iperf3 -s`
2. En el PC cliente: `iperf3 -c {IP_servidor} -t 30`
3. Repita la prueba en banda 2,4GHz y 5 GHz. Registre el throughput promedio y la varianza.
4. Repita variando la distancia al AP: 1 m, 5 m, 10 m. Registre el RSSI con `iw dev wlan0 station dump`.

4.4.5 Paso 5 — Captura del handshake WPA2

1. Con Wireshark en modo monitor, aplique el filtro `eapol` para capturar el proceso de autenticación.
2. Desconecte y reconecte un cliente de la red.
3. Identifique los cuatro mensajes del 4-way handshake y documente la información contenida en cada mensaje.

4.5 Preguntas de Análisis

- P1.** ¿Qué diferencias observa entre las tramas capturadas en 2,4 GHz y 5 GHz en cuanto a throughput, latencia e interferencias?
- P2.** Explique el proceso de asociación 802.11 identificando cada tipo de trama (Probe, Auth, Assoc).
- P3.** ¿Qué información expone una trama Beacon de una red abierta? ¿Qué riesgos de privacidad implica?
- P4.** ¿Qué es el 4-way handshake de WPA2? ¿Por qué es crítico para la seguridad de la red?
- P5.** ¿Cuáles son las ventajas de WPA3 frente a WPA2? ¿Qué limitaciones de compatibilidad puede presentar?

4.6 Entregables

Entregables

1. Informe técnico con capturas de Wireshark anotadas.
2. Tabla comparativa de redes detectadas (con análisis de seguridad).
3. Gráfica throughput vs. distancia para ambas bandas.
4. Archivo `.pcapng` con la captura del handshake WPA2.

5. Repositorio con scripts de automatización (si aplica).

4.7 Rúbrica de Evaluación

Cuadro 4.4: Rúbrica — Práctica P-C4 (10 puntos)

Criterio	Peso	Indicadores de logro
Presentación y defensa	20 %	Expone con claridad el procedimiento y los resultados; responde preguntas técnicas.
Informe escrito	20 %	Capturas correctamente anotadas; análisis de tramas preciso; bibliografía IEEE.
Calidad técnica	60 %	Captura ≥ 10 redes; análisis correcto de Beacon y handshake; gráfica throughput-distancia; comparativa 2,4/5 GHz.

Capítulo 5

P-C5: Comunicación Bluetooth Low Energy entre ESP32

Unidad:	400 – Bluetooth / IEEE 802.15	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

5.1 Marco Teórico

5.1.1 Bluetooth y el estándar IEEE 802.15

Bluetooth opera en la banda ISM de 2,4 GHz y utiliza la técnica de *frequency hopping spread spectrum* (FHSS) entre 79 canales de 1 MHz. El estándar IEEE 802.15 agrupa tecnologías de redes de área personal inalámbricas (WPAN).

5.1.2 Bluetooth Low Energy (BLE)

BLE (Bluetooth 4.0+, 2010) es un subconjunto optimizado para dispositivos con restricción de energía. Sus características principales:

- Canales de *advertising*: 3 canales dedicados (37, 38, 39) para difusión de presencia.
- Arquitectura GATT (*Generic ATtribute Profile*): organiza los datos en **Services** y **Characteristics** identificados por UUIDs de 128 bits.
- Roles: *Central* (escanea y conecta) y *Peripheral* (anuncia y acepta conexiones).
- Consumo típico: 0,01–0,5 mA en reposo; picos de 15 mA durante transmisión.

5.1.3 BLE en el ESP32

El ESP32 soporta BLE 4.2 mediante la pila de bajo nivel *Bluedroid* de Espressif, accesible desde ESP-IDF mediante la pila **NimBLE** (recomendada) o *Bluedroid*.

5.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Implementar un enlace BLE entre dos ESP32 en modo Central-Peripheral, transmitiendo datos de sensores en tiempo real.

Objetivos específicos

- OE1.** Configurar un ESP32 como servidor BLE (Peripheral) que exponga una característica con datos del sensor DHT11 (temperatura y humedad).
- OE2.** Configurar un segundo ESP32 como cliente BLE (Central) que lea la característica y muestre los datos en el Monitor Serie.
- OE3.** Medir el alcance efectivo del enlace BLE y la tasa de actualización de datos.
- OE4.** Comparar el consumo de energía de BLE frente a Wi-Fi para la misma tarea.

5.3 Materiales y Equipos

Materiales y Equipos

Cant.	Componente	Ref.	Observaciones
2	Placa NodeMCU ESP32	ESP32-WROOM-32	Una actúa de Peripheral, otra de Central
2	Sensor de temperatura/humedad	DHT11 o DHT22	
2	Resistencia 10 kΩ	—	Pull-up datos DHT
1	Multímetro o analizador	—	Medición corriente
—	Cables Dupont	—	
2	Cable USB tipo-C	—	

Software: VSCode, extensión **Espressif IDF**, ESP-IDF ≥ 5.1 , NimBLE (built-in), comp. espressif/esp-idf-dht.

5.4 Procedimiento

5.4.1 Paso 0 — Configuración del entorno de desarrollo

Siga los pasos 1–4 de P-C2 (VSCode + extensión Espressif IDF + ESP-IDF v5.x). Cree el proyecto con nombre `ble_dht` y chip ESP32. En `main/idf_component.yml` agregue:

```
1 dependencies:
2   idf: ">=5.1"
3   espressif/esp-idf-dht: "*"
```

Listing 5.1: `main/idf_component.yml` — P-C5 BLE

La pila **NimBLE** está incluida en ESP-IDF; actívela en `menuconfig`: *Component config* → *Bluetooth* → *NimBLE*. Compile y cargue con *ESP-IDF: Build* (`Ctrl+E, B`) y *ESP-IDF: Flash* (`Ctrl+E, F`).

5.4.2 Paso 1 — ESP32 Peripheral (servidor BLE)

```
1 #include <BLEDevice.h>
2 #include <BLEServer.h>
3 #include <BLEUtils.h>
4 #include <BLE2902.h>
5 #include <DHT.h>
6
7 #define DHT_PIN 4
8 #define DHT_TYPE DHT11
9
10 // UUIDs del servicio y característica
11 #define SERVICE_UUID "12345678-1234-1234-1234-123456789abc"
12 #define CHAR_UUID "abcd1234-ab12-ab12-ab12-abcdef123456"
13
14 DHT dht(DHT_PIN, DHT_TYPE);
15 BLECharacteristic* pCharacteristic;
16 bool deviceConnected = false;
17
18 class MyServerCallbacks : public BLEServerCallbacks {
19   void onConnect(BLEServer* s) { deviceConnected = true; }
20   void onDisconnect(BLEServer* s) { deviceConnected = false; }
21 };
22
23 void setup() {
24   Serial.begin(115200);
25   dht.begin();
26
27   BLEDevice::init("ESP32-Sensor");
28   BLEServer* pServer = BLEDevice::createServer();
29   pServer->setCallbacks(new MyServerCallbacks());
30
31   BLEService* pService = pServer->createService(SERVICE_UUID);
32   pCharacteristic = pService->createCharacteristic(
33     CHAR_UUID,
```

```

34     BLECharacteristic::PROPERTY_READ |
        BLECharacteristic::PROPERTY_NOTIFY
35 );
36 pCharacteristic->addDescriptor(new BLE2902());
37 pService->start();
38
39 BLEAdvertising* pAdv = BLEDevice::getAdvertising();
40 pAdv->addServiceUUID(SERVICE_UUID);
41 pAdv->start();
42 Serial.println("BLE Peripheral listo.");
43 }
44
45 void loop() {
46     if (deviceConnected) {
47         float temp = dht.readTemperature();
48         float hum = dht.readHumidity();
49
50         if (!isnan(temp) && !isnan(hum)) {
51             String datos = "T:" + String(temp,1) + " H:" + String(hum,1);
52             pCharacteristic->setValue(datos.c_str());
53             pCharacteristic->notify();
54             Serial.println("Enviado: " + datos);
55         }
56     }
57     delay(2000);
58 }

```

Listing 5.2: ESP32 como servidor BLE con datos DHT11

5.4.3 Paso 2 — ESP32 Central (cliente BLE)

```

1  #include <BLEDevice.h>
2  #include <BLEClient.h>
3
4  #define SERVICE_UUID "12345678-1234-1234-1234-123456789abc"
5  #define CHAR_UUID    "abcd1234-ab12-ab12-ab12-abcdef123456"
6
7  BLEClient* pClient;
8  BLERemoteCharacteristic* pRemoteChar;
9  bool connected = false;
10
11 static void notifyCallback(BLERemoteCharacteristic* c,
12     uint8_t* data, size_t len, bool isNotify) {
13     String msg = "";
14     for (int i = 0; i < len; i++) msg += (char)data[i];
15     Serial.println("Recibido: " + msg);
16 }
17
18 void setup() {

```

```
19 Serial.begin(115200);
20 BLEDevice::init("");
21
22 BLEScan* pScan = BLEDevice::getScan();
23 BLEScanResults results = pScan->start(5, false);
24
25 for (int i = 0; i < results.getCount(); i++) {
26     BLEAdvertisedDevice dev = results.getDevice(i);
27     if (dev.getName() == "ESP32-Sensor") {
28         pClient = BLEDevice::createClient();
29         pClient->connect(&dev);
30         BLERemoteService* pSvc =
31             pClient->getService(BLEUUID(SERVICE_UUID));
32         if (pSvc) {
33             pRemoteChar = pSvc->getCharacteristic(BLEUUID(Char_UUID));
34             if (pRemoteChar->canNotify())
35                 pRemoteChar->registerForNotify(notifyCallback);
36             connected = true;
37             Serial.println("Conectado al Peripheral.");
38         }
39         break;
40     }
41 }
42 }
43
44 void loop() { delay(100); }
```

Listing 5.3: ESP32 como cliente BLE

5.4.4 Paso 3 — Medición de alcance

Mueva el ESP32 Central a diferentes distancias del Peripheral (1, 5, 10, 15, 20 m) y en distintas condiciones (línea de vista, con paredes intermedias). Para cada posición registre:

- RSSI obtenido con `pClient->getRssi()`.
- Tasa de recepción de notificaciones (mensajes/min).
- Ocurrencia de desconexiones.

5.5 Preguntas de Análisis

- P1.** ¿Qué diferencias existen entre BLE Classic y BLE en cuanto a velocidad, consumo y casos de uso?
- P2.** Explique la arquitectura GATT: Services, Characteristics, Descriptors y cómo se identifican mediante UUIDs.

- P3.** ¿Cómo afectan los obstáculos físicos al alcance de BLE? ¿Qué estrategias de mitigación existen?
- P4.** ¿Qué mecanismos de seguridad ofrece BLE 4.2? ¿Son suficientes para una aplicación IoT médica?
- P5.** Compare el consumo energético de BLE vs. Wi-Fi para transmitir los mismos datos cada 2 segundos.

5.6 Entregables

Entregables

1. Informe técnico con códigos comentados.
2. Gráfica RSSI vs. distancia en línea de vista y con obstáculos.
3. Tabla de tasa de recepción vs. distancia.
4. Análisis comparativo BLE vs. Wi-Fi (consumo y latencia).
5. Repositorio Git con ambos sketches.

5.7 Rúbrica de Evaluación

Cuadro 5.1: Rúbrica — Práctica P-C5 (10 puntos)

Criterio	Peso	Indicadores de logro
Presentación y defensa	20 %	Demuestra el enlace BLE en vivo; responde preguntas sobre GATT.
Informe escrito	20 %	Código documentado; gráficas claras; bibliografía correcta.
Calidad técnica	60 %	Enlace Central-Peripheral funcional; datos DHT recibidos correctamente; curva RSSI-distancia completa; análisis energético.

Capítulo 6

P-C6: Red de Sensores con ZigBee / IEEE 802.15.4

Unidad:	500 – ZigBee / IEEE 802.15.4	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

6.1 Marco Teórico

6.1.1 El estándar IEEE 802.15.4

El estándar **IEEE 802.15.4** define la capa física (PHY) y el control de acceso al medio (MAC) para redes inalámbricas de área personal de baja tasa (*LR-WPAN*). Sus características principales son:

Cuadro 6.1: Características del estándar IEEE 802.15.4

Parámetro	Valor
Bandas de frecuencia	868 MHz (1 canal), 915 MHz (10 canales), 2,4 GHz (16 canales)
Velocidad de datos	250 kbps (2,4 GHz), 40 kbps (915 MHz)
Rango típico	10–100 m (interiores)
Topologías soportadas	Estrella, árbol de clusters, malla (<i>mesh</i>)
Consumo energético	Modo activo ≈ 30 mA; modo <i>sleep</i> $< 1 \mu\text{A}$
Direccionamiento	64-bit IEEE + 16-bit corta
Seguridad	AES-128 en capa MAC

6.1.2 ZigBee sobre IEEE 802.15.4

ZigBee es un protocolo de red y aplicación construido sobre la capa física y MAC de IEEE 802.15.4. Es gestionado por la *ZigBee Alliance* (actual Connectivity Standards Alliance, CSA) y define:

- **Capa de red (NWK):** enrutamiento en malla con el algoritmo AODV modificado; direccionamiento jerárquico de árbol.
- **Capa de aplicación (APL):** incluye el *Application Support Sub-layer* (APS) y el *ZigBee Device Object* (ZDO).
- **Perfiles de aplicación:** Home Automation (HA), Smart Energy (SE), ZigBee PRO.

6.1.3 Roles de dispositivos ZigBee

Cuadro 6.2: Roles de dispositivos en una red ZigBee

Rol	IEEE 802.15.4	Función ZigBee
Coordinador (ZC)	FFD	Único por red; forma y gestiona la PAN; almacena claves de seguridad.
Router (ZR)	FFD	Extiende la cobertura; enruta paquetes; siempre activo.
End Device (ZED)	RFD	Dispositivo final (sensor/actuador); puede dormir para ahorrar energía.

FFD = Full Function Device; RFD = Reduced Function Device.

6.1.4 Módulos XBee

Los módulos **XBee** de Digi International implementan IEEE 802.15.4 y/o ZigBee PRO en un factor de forma estándar de 2,54 mm. Los modelos más comunes en laboratorio son:

Cuadro 6.3: Modelos XBee frecuentes en entornos educativos

Modelo	Protocolo	Potencia TX	Alcance urbano
XBee Series 1 (S1)	IEEE 802.15.4	1 mW (0 dBm)	≈30 m
XBee Series 2 (S2C)	ZigBee PRO	2 mW (3 dBm)	≈60 m
XBee 3	ZigBee / 802.15.4 / DigiMesh	6,3 mW (8 dBm)	≈90 m

6.1.5 Estructura de la trama IEEE 802.15.4

La trama de datos IEEE 802.15.4 contiene:

1. **Preámbulo** (4 bytes) + **SFD** (1 byte): sincronización.
2. **PHR** (1 byte): longitud del PSDU.
3. **MHR**: Frame Control (2), Sequence Number (1), direcciones (hasta 20 bytes).
4. **Payload** (hasta 102 bytes).
5. **MFR**: CRC-16 (2 bytes).

6.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Implementar una red de sensores ZigBee con tres nodos (Coordinador, Router y End Device) transmitiendo datos de temperatura en tiempo real a través de una topología en estrella y en malla.

Objetivos específicos

- OE1.** Configurar los roles Coordinador, Router y End Device en módulos XBee Series 2 mediante XCTU.
- OE2.** Implementar la comunicación punto a punto y punto a multipunto entre los nodos de la red.
- OE3.** Leer datos del sensor DHT22 en el End Device y transmitirlos al Coordinador.
- OE4.** Analizar tramas ZigBee capturadas con XCTU o Wireshark y verificar el enrutamiento en malla.

6.3 Materiales y Equipos

Materiales y Equipos

Cant.	Componente	Referencia	Observaciones
3	Módulo XBee Series 2 (S2C)	XB24CZ7WIT-004	Uno por rol
3	SparkFun XBee Explorer USB	WRL-11812	Adaptador serial-USB
1	Sensor DHT22	AM2302	Para el End Device
1	Resistencia 10 k Ω	—	Pull-up datos DHT22
3	Cable USB tipo-A a micro-B	—	Conexión PC-Shield
1	Computador con XCTU	$\geq$ v6.5	https://hub.digi.com/support/products/xctu/
1	Placa NodeMCU ESP32	ESP32-WROOM-32	Nodo sensor (End Device)

6.4 Procedimiento

6.4.1 Paso 1 — Instalación y configuración con XCTU

1. Instale **XCTU** y conecte el primer módulo XBee vía USB.
2. Haga clic en *Add devices* para detectar el módulo. Verifique la versión de firmware; actualice si es necesario a **ZigBee Coordinator API** (función: Coordinator, modo: API).
3. Configure los parámetros mínimos para el Coordinador:

Cuadro 6.4: Parámetros de configuración XBee – Coordinador

Parámetro	Código AT	Valor sugerido	Descripción
PAN ID	ID	1234	Identificador de red
Channel	CH	C (12)	Canal 2,4 GHz
CE (Coord.)	CE	1	Habilita rol Coordinador
AP (API)	AP	2	API mode with escaping
Baud rate	BD	7 (115200)	Velocidad serial
Node ID	NI	COORD	Nombre del nodo

4. Repita para el segundo módulo con rol **Router** (firmware: ZigBee Router API, CE=0, NI=ROUTER).
5. Repita para el tercer módulo con rol **End Device** (firmware: ZigBee End Device API, SM=4 para sleep periódico, NI=SENSOR).
6. Asegúrese de que los tres módulos compartan el mismo **PAN ID**.

i Verificación de asociación

En XCTU, la pantalla de *Network* muestra la topología de red en tiempo real. El LED **ASSOC** del módulo parpadea lentamente (1 Hz) cuando está asociado a la PAN. Un parpadeo rápido indica que aún busca red.

6.4.2 Paso 2 — Comunicación punto a punto (transparente)

1. Cambie temporalmente los módulos a modo AT (AP=0).
2. Configure la dirección destino del Router con la dirección de 64-bit del Coordinador (DH y DL).
3. Use la consola serial de XCTU para enviar mensajes de texto desde el Router al Coordinador y verifique la recepción.
4. Registre la latencia (tiempo entre envío y recepción) para 10 mensajes consecutivos.

6.4.3 Paso 3 — Lectura del sensor DHT22 con ESP32 (End Device)

El ESP32 lee el sensor DHT22 y reenvía los datos al módulo XBee End Device vía UART2.

```

1 #include <stdio.h>
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "driver/uart.h"
5 #include "driver/gpio.h"
6 #include "esp_log.h"
7 #include "dht.h"           // espressif/esp-idf-dht
8
9 #define DHT_GPIO    GPIO_NUM_4
10 #define DHT_TYPE    DHT_TYPE_DHT22
11 #define XBEE_UART   UART_NUM_2
12 #define XBEE_TX      GPIO_NUM_17
13 #define XBEE_RX      GPIO_NUM_16
14 #define BAUD_RATE   9600
15
16 static const char *TAG = "SENSOR";
17
18 static void xbee_uart_init(void) {
19     uart_config_t cfg = {
20         .baud_rate    = BAUD_RATE,
21         .data_bits    = UART_DATA_8_BITS,
22         .parity        = UART_PARITY_DISABLE,
23         .stop_bits    = UART_STOP_BITS_1,
24         .flow_ctrl    = UART_HW_FLOWCTRL_DISABLE,
25     };
26     uart_driver_install(XBEE_UART, 256, 0, 0, NULL, 0);
27     uart_param_config(XBEE_UART, &cfg);
28     uart_set_pin(XBEE_UART, XBEE_TX, XBEE_RX,
29                 UART_PIN_NO_CHANGE, UART_PIN_NO_CHANGE);
30 }
31
32 void app_main(void) {
33     xbee_uart_init();
34     ESP_LOGI(TAG, "Nodo sensor listo.");
35
36     while (1) {
37         float temp, hum;
38         if (dht_read_float_data(DHT_TYPE, DHT_GPIO,
39                                &hum, &temp) == ESP_OK) {
40             char buf[32];
41             int n = snprintf(buf, sizeof(buf),
42                             "T: %.1f,H: %.1f\r\n", temp, hum);
43             uart_write_bytes(XBEE_UART, buf, n);
44             ESP_LOGI(TAG, "TX -> %.*s", n - 2, buf);
45         } else {
46             ESP_LOGW(TAG, "Error al leer DHT22");
47         }
48     }
49 }

```

```

48     vTaskDelay(pdMS_TO_TICKS(5000));
49     }
50 }

```

Listing 6.1: ESP32 + DHT22 transmitiendo por XBee (modo AT) — ESP-IDF

6.4.4 Paso 4 — Recepción en el Coordinador

Conecte el Coordinador al PC y abra el Monitor Serie (9600 bps) o la consola XCTU. Verifique que los datos del sensor aparecen correctamente. Registre al menos 20 muestras consecutivas y calcule:

- **Tasa de entrega de paquetes (PDR):** $PDR = \frac{N_{\text{recibidos}}}{N_{\text{enviados}}} \times 100 \%$
- **Latencia promedio** entre envío y recepción.
- **RSSI promedio** reportado por el Coordinador.

6.4.5 Paso 5 — Topología en malla

1. Coloque el Router entre el End Device y el Coordinador, fuera del alcance directo del End Device (verifique que el RSSI directo sea inferior a -80 dBm).
2. Confirme en XCTU Network que el End Device enruta sus paquetes a través del Router.
3. Compare el PDR antes y después de introducir el Router como nodo intermedio.

i Topología en XCTU Network

La vista *Network* de XCTU muestra los enlaces activos con el RSSI de cada salto. Use *Scan* periódico para actualizar la topología.

6.5 Preguntas de Análisis

- P1.** ¿Qué diferencias existen entre IEEE 802.15.4 y ZigBee? ¿Cuáles capas del modelo OSI cubre cada uno?
- P2.** Explique el proceso de *association* mediante el que un End Device se une a la PAN. ¿Qué tramas intercambia con el Coordinador?
- P3.** ¿Por qué el End Device puede dormir ($SM=4$) pero el Router no? ¿Qué implicaciones tiene esto en el diseño de la red?
- P4.** Analice el consumo energético teórico de un End Device ZigBee que transmite una muestra cada 60 segundos con una batería de 2 Ah a 3 V. ¿Cuánto tiempo podría operar de forma autónoma?
- P5.** Compare ZigBee con BLE Mesh y Z-Wave para una aplicación de domótica residencial con 30 nodos. ¿Cuál elegiría y por qué?

6.6 Entregables

Entregables

1. Informe técnico con marco teórico completo y análisis de resultados.
2. Captura de la topología de red desde XCTU Network (estrella y malla).
3. Tabla de PDR, RSSI y latencia para ambas topologías.
4. Código ESP-IDF comentado (`main.c` completo).
5. Cálculo de autonomía energética del End Device.
6. Repositorio Git con todos los archivos del proyecto.

6.7 Rúbrica de Evaluación

Cuadro 6.5: Rúbrica — Práctica P-C6 (10 puntos)

Criterio	Peso	Indicadores de logro
Presentación y defensa	20 %	Demuestra la red ZigBee en vivo; explica los roles de dispositivos y el proceso de asociación; responde preguntas sobre IEEE 802.15.4.
Informe escrito	20 %	Capturas de XCTU Network anotadas; tabla de métricas completa; código documentado; bibliografía IEEE.
Calidad técnica	60 %	Red de tres nodos funcional en topología estrella y malla; sensor DHT22 transmitiendo correctamente; PDR calculado para ≥ 20 muestras en ambas topologías; cálculo de autonomía energética realizado.

Capítulo 7

P-C7: Enlace LoRa entre Gateway y Mote

Unidad:	600 – Tecnologías de cobertura amplia	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

7.1 Marco Teórico

7.1.1 LoRa y LoRaWAN

LoRa (*Long Range*) es una técnica de modulación de espectro ensanchado basada en *Chirp Spread Spectrum* (CSS), desarrollada por Semtech. Permite comunicaciones de **baja potencia y largo alcance** (hasta 15 km en zona rural, 2–5 km en zona urbana) con tasas de datos de 0,3 a 50 kbps.

LoRaWAN es el protocolo de red MAC que opera sobre la capa física LoRa. Define tres clases de dispositivos:

Cuadro 7.1: Clases de dispositivos LoRaWAN

Clase	Recepción	Aplicación típica
A	Solo tras transmisión (2 ventanas)	Sensores IoT de bajo consumo (meteorológicos, agrícolas)
B	Ventanas programadas (beacon)	Actuadores que requieren latencia predecible
C	Recepción continua	Dispositivos alimentados por red (medidores inteligentes)

7.1.2 Parámetros de enlace LoRa

Los parámetros de la capa física determinan el balance entre alcance, velocidad y consumo:

Cuadro 7.2: Parámetros físicos de LoRa y su efecto

Parámetro	Rango típico	Efecto
Spreading Factor (SF)	SF7–SF12	SF mayor $\Rightarrow$ mayor alcance, menor velocidad, mayor consumo de airtime

Bandwidth (BW)	125 / 250 / 500 kHz	BW mayor $\Rightarrow$ mayor velocidad, menor sensibilidad
Coding Rate (CR)	4/5 – 4/8	CR mayor $\Rightarrow$ mayor robustez ante errores, menor eficiencia
TX Power	2–20 dBm	Potencia mayor $\Rightarrow$ mayor alcance, mayor consumo

7.1.3 Ecuación de enlace de Friis

La pérdida de trayecto en espacio libre se calcula con:

$$L_{fs} [\text{dB}] = 20 \log_{10}(d) + 20 \log_{10}(f) + 20 \log_{10}\left(\frac{4\pi}{c}\right) \quad (7.1)$$

donde d es la distancia en metros y f la frecuencia en Hz. El presupuesto de enlace (*link budget*) determina la distancia máxima alcanzable:

$$\text{Link budget} [\text{dB}] = P_{\text{TX}} + G_{\text{TX}} + G_{\text{RX}} - L_{\text{cable}} - \text{NF} - \text{SNR}_{\text{mín}} \quad (7.2)$$

7.1.4 Arquitectura de red LoRaWAN

- **Mote (End Device):** sensor/actuador que transmite paquetes LoRa.
- **Gateway LoRa:** recibe todos los paquetes del espectro radioeléctrico y los reenvía a un servidor de red vía IP (Ethernet / Wi-Fi / LTE).
- **Network Server:** gestiona la topología, la deduplicación de paquetes y la seguridad (AES-128).
- **Application Server:** procesa los datos de la aplicación.

La red LoRaWAN más utilizada a nivel mundial es **The Things Network (TTN)**, de acceso comunitario y gratuito.

7.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Implementar un enlace punto a punto entre un *Mote* LoRa (ESP32 + módulo LoRa) y un *Gateway* LoRa, evaluando parámetros de enlace y consumo energético.

Objetivos específicos

- OE1.** Configurar el Mote para transmitir datos de sensores con distintos Spreading Factors.
- OE2.** Medir el RSSI, SNR y distancia máxima alcanzada para cada SF.

OE3. Registrar los paquetes recibidos en el Gateway y calcular el packet delivery ratio (PDR).

OE4. Calcular el presupuesto de enlace teórico y compararlo con los valores medidos.

7.3 Materiales y Equipos

✂ Materiales y Equipos

Cant.	Componente	Ref.	Observaciones
2	Placa ESP32 + módulo LoRa	Heltec WiFi LoRa 32 v3 o TTGO LoRa32	Incluye OLED
1	Antena LoRa 868/915 MHz	—	Dipolo 2 dBi
1	Computador	—	Configuración y logs
2	Cable USB tipo-C	—	
1	Batería LiPo 3,7V	—	Alimentación Mote en campo

Software: VSCode, extensión **Espressif IDF**, ESP-IDF ≥ 5.1 , lib. **RadioLib** (jgromes, soporte nativo ESP-IDF), TTN Console.

7.4 Procedimiento

7.4.1 Paso 0 — Configuración del entorno de desarrollo

Siga los pasos 1–4 de P-C2 (VSCode + extensión Espressif IDF + ESP-IDF v5.x). Cree el proyecto con nombre `lora_enlace` y chip según la placa: **esp32** para TTGO LoRa32, **esp32s3** para Heltec LoRa 32 v3.

Agregue RadioLib como componente (soporta ESP-IDF nativo desde v6.x):

```
1 mkdir -p components && cd components
2 git clone --depth 1 \
3 https://github.com/jgromes/RadioLib.git RadioLib
```

Listing 7.1: Agregar RadioLib como componente ESP-IDF

En `main/idf_component.yml`:

```
1 dependencies:
2   idf: ">=5.1"
```

Listing 7.2: main/idf_component.yml — P-C7 LoRa

Compile y cargue con *ESP-IDF: Build* (Ctrl+E, B) y *ESP-IDF: Flash* (Ctrl+E, F).

7.4.2 Paso 1 — Configuración del Mote (transmisor)

```
1 #include <RadioLib.h>
2
3 // Pines para Heltec WiFi LoRa 32 v3
4 SX1262 radio = new Module(8, 14, 12, 13);
5
6 int counter = 0;
7
8 void setup() {
9   Serial.begin(115200);
10
11   Serial.print("Iniciando LoRa... ");
12   int state = radio.begin(
13     868.1, // Frecuencia (MHz) - Europa 868 / USA 915
14     125.0, // Bandwidth (kHz)
15     9,     // Spreading Factor (7-12)
16     7,     // Coding Rate (5 = 4/5)
17     0x12,  // Sync word LoRaWAN publica
18     14,    // TX Power (dBm)
19     8      // Preamble length
20   );
21
22   if (state == RADIOLIB_ERR_NONE) {
23     Serial.println("OK");
24   } else {
25     Serial.print("Error: "); Serial.println(state);
26     while (true);
27   }
28 }
29
30 void loop() {
31   String payload = "Paquete #" + String(counter++);
32   Serial.print("Enviando: " + payload + " ... ");
33
34   int state = radio.transmit(payload);
35   if (state == RADIOLIB_ERR_NONE) {
36     Serial.println("OK");
37   } else {
38     Serial.print("Error: "); Serial.println(state);
39   }
40 }
```



```
41 delay(5000); // Respetar duty cycle 1% en banda 868 MHz
42 }
```

Listing 7.3: Mote LoRa con RadioLib — transmisión periódica

7.4.3 Paso 2 — Configuración del Gateway (receptor)

```
1 #include <RadioLib.h>
2
3 SX1262 radio = new Module(8, 14, 12, 13);
4
5 void setup() {
6   Serial.begin(115200);
7   radio.begin(868.1, 125.0, 9, 7, 0x12, 14, 8);
8   Serial.println("Gateway listo. Esperando paquetes...");
9 }
10
11 void loop() {
12   String received;
13   int state = radio.receive(received);
14
15   if (state == RADIOLIB_ERR_NONE) {
16     Serial.println("=== PAQUETE RECIBIDO ===");
17     Serial.println("Datos   : " + received);
18     Serial.print("RSSI     : "); Serial.print(radio.getRSSI());
19     Serial.println(" dBm");
20     Serial.print("SNR      : "); Serial.print(radio.getSNR());
21     Serial.println(" dB");
22     Serial.print("FreqErr : ");
23     Serial.print(radio.getFrequencyError());
24     Serial.println(" Hz");
25 }
```

Listing 7.4: Gateway LoRa — recepción continua con métricas

7.4.4 Paso 3 — Medición de enlace vs. Spreading Factor

Repita la prueba con SF = 7, 9, 10 y 12. Para cada SF:

1. Transmita 20 paquetes consecutivos y registre cuántos llegan al Gateway (PDR).
2. Registre el RSSI y SNR promedio.
3. Calcule el *Time on Air* (ToA) con la fórmula de Semtech o el calculador <https://www.thethingsnetwork.org/airtime-calculator>.
4. Mueva el Mote a 50 m, 100 m, 200 m y registre si el PDR cambia.

7.4.5 Paso 4 — Presupuesto de enlace

Complete la Tabla 7.3 con los valores de su equipo:

Cuadro 7.3: Plantilla de presupuesto de enlace LoRa

Parámetro	Valor (dB/dBm)	Fuente
Potencia TX		Configuración sketch
Ganancia antena TX		Datasheet antena
Pérdida cable TX		Estimación
Ganancia antena RX		Datasheet antena
Sensibilidad RX (SF9, BW125)		Datasheet SX1262
Link budget total		Ecuación 7.2
Distancia máx. teórica		Ecuación 7.1
Distancia máx. medida		Experimento

7.5 Preguntas de Análisis

- P1.** ¿Cómo afecta el Spreading Factor al alcance, a la velocidad de datos y al consumo energético? Explique el compromiso (*trade-off*) involucrado.
- P2.** ¿Qué es el *duty cycle* y por qué es una restricción regulatoria en la banda 868 MHz en Europa?
- P3.** Compare LoRa con NB-IoT y Sigfox en cuanto a ancho de banda, latencia, costo de infraestructura y escalabilidad.
- P4.** ¿Qué mecanismos de seguridad implementa LoRaWAN? Identifique las claves NwkS-Key y AppSKey y su función.
- P5.** ¿Cuáles son las principales aplicaciones de LoRa en entornos urbanos y agrícolas? Proponga un caso de uso en Ecuador.

7.6 Entregables

Entregables

1. Informe técnico con marco teórico y resultados.
2. Tabla PDR, RSSI y SNR para cada SF y distancia.
3. Tabla de presupuesto de enlace (teórico vs. medido).

4. Gráfica PDR vs. distancia para SF7 y SF12.
5. Repositorio Git con ambos sketches (Mote y Gateway).

7.7 Rúbrica de Evaluación

Cuadro 7.4: Rúbrica — Práctica P-C7 (10 puntos)

Criterio	Peso	Indicadores de logro
Presentación y defensa	20 %	Demuestra el enlace LoRa en vivo; explica el efecto del SF.
Informe escrito	20 %	Gráficas correctas; presupuesto de enlace completo; bibliografía IEEE.
Calidad técnica	60 %	Enlace Gateway-Mote funcional; medición de PDR para ≥ 3 SF y ≥ 3 distancias; cálculo correcto del link budget; comparativa teórico-medido.

Capítulo 8

P-C8: Auditoría de Seguridad en Redes Wi-Fi

Unidad:	700 – Seguridad en redes inalámbricas	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

Alcance legal y ético

Esta práctica se realiza **exclusivamente** sobre redes Wi-Fi propias o sobre el laboratorio de práctica habilitado por el docente. Auditar redes ajenas sin autorización escrita es un delito tipificado en el Código Orgánico Integral Penal (COIP) del Ecuador (Art. 234). El incumplimiento implica calificación de cero y reporte a las autoridades correspondientes.

8.1 Marco Teórico

8.1.1 Amenazas en redes inalámbricas

Las redes Wi-Fi son especialmente susceptibles a:

- **Evil Twin / AP falso:** un atacante crea un AP con el mismo SSID que el legítimo para interceptar tráfico (*man-in-the-middle*).
- **Deauthentication attack:** inyección de tramas de desautenticación (802.11 management frames no autenticadas) para desconectar clientes y capturar el handshake WPA2.
- **Offline password cracking:** el handshake WPA2 capturado se ataca mediante diccionario o fuerza bruta sin necesidad de conexión a la red.
- **PMKID attack:** permite obtener el hash de la clave WPA2 sin necesidad de que ningún cliente se conecte.
- **Rogue DHCP:** un servidor DHCP malicioso asigna una puerta de enlace falsa redirigiendo el tráfico.

8.1.2 Herramientas de auditoría

Cuadro 8.1: Herramientas de auditoría Wi-Fi (uso en entornos autorizados)

Herramienta	Función	Observaciones
-------------	---------	---------------

airmon-ng	Activar modo monitor	Suite aircrack-ng
airodump-ng	Escaneo y captura de tramas	Incluye handshake
aireplay-ng	Inyección de tramas (deauth)	Requiere autorización
aircrack-ng	Cracking de clave WPA2 (diccionario)	
hashcat	Cracking GPU de WPA2/PMKID	Más rápido que CPU
Wireshark	Análisis de paquetes	
nmap	Escaneo de hosts y puertos	

8.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Realizar una auditoría controlada sobre una red Wi-Fi de laboratorio, identificando vulnerabilidades y proponiendo medidas de hardening.

Objetivos específicos

- OE1.** Identificar amenazas activas en el entorno de laboratorio mediante escaneo pasivo.
- OE2.** Capturar el handshake WPA2 de la red de práctica mediante desautenticación controlada.
- OE3.** Evaluar la fortaleza de la contraseña con un ataque de diccionario.
- OE4.** Documentar vulnerabilidades encontradas y proponer un plan de hardening.

8.3 Materiales y Equipos

Materiales y Equipos

Cant.	Componente	Ref.	Observaciones
1	Máquina virtual Kali Linux	≥ 2024.1	O instalación nativa
1	Adaptador Wi-Fi USB	Alfa AWUSO36ACH	Chip rtl8812au, soporte inyección
1	AP de práctica	TP-Link o similar	Provisto por el laboratorio
1	Wordlist	rockyou.txt	Incluida en Kali: /usr/share/wordlists/

8.4 Procedimiento

8.4.1 Paso 1 — Preparación del entorno Kali

```
1 # Identificar interfaz Wi-Fi
2 iwconfig
3
4 # Activar modo monitor
5 sudo airmon-ng start wlan0
6
7 # Verificar
8 iwconfig wlan0mon
```

Listing 8.1: Preparación del adaptador en modo monitor

8.4.2 Paso 2 — Escaneo de redes

```
1 # Escanear todas las redes visibles
2 sudo airodump-ng wlan0mon
3
4 # Fijar canal y BSSID de la red de practica, guardar captura
5 sudo airodump-ng -c <CANAL> --bssid <BSSID> \
6     -w captura wlan0mon
7
8 # En otra terminal: enviar tramas deauth al AP (maximo 5)
9 sudo aireplay-ng --deauth 5 -a <BSSID> wlan0mon
```

Listing 8.2: Escaneo con airodump-ng y captura de handshake

Espere a que airodump-ng muestre WPA handshake: en la esquina superior derecha.

8.4.3 Paso 3 — Ataque de diccionario

```
1 # Intentar recuperar la clave con rockyou.txt
2 aircrack-ng captura-01.cap \
3     -w /usr/share/wordlists/rockyou.txt.gz
4
5 # Verificar resultado
6 echo "Clave encontrada: <clave>"
```

Listing 8.3: Ataque de diccionario con aircrack-ng

i Nota didáctica

Si la contraseña de práctica no está en el diccionario (resultado esperado en un sistema bien configurado), esto confirma la importancia de usar contraseñas fuertes. Documente el tiempo de ataque y el número de claves probadas por segundo.

8.4.4 Paso 4 — Plan de hardening

Complete la siguiente tabla con las vulnerabilidades identificadas y las contramedidas propuestas:

Cuadro 8.2: Plantilla de análisis de vulnerabilidades y hardening

Vulnerabilidad	Riesgo	Contramedida propuesta
Contraseña débil		
WPA2 (sin WPA3)		
SSID difundido		
PMF desactivado		

8.5 Preguntas de Análisis

- P1.** Explique el proceso completo de un ataque de captura de handshake WPA2. ¿Qué trama 802.11 se inyecta y por qué?
- P2.** ¿Por qué una contraseña de ocho caracteres puede ser vulnerada en minutos con una GPU moderna? ¿Qué longitud mínima y qué características debe tener una contraseña segura para WPA2?
- P3.** ¿Qué es el Management Frame Protection (PMF/802.11w) y cómo mitiga los ataques de desautenticación?
- P4.** ¿Cómo implementaría WPA3 en el AP de laboratorio? ¿Qué ventajas ofrece el modo de transición WPA2/WPA3?
- P5.** Proponga una política de seguridad Wi-Fi para una empresa con 50 empleados y red de invitados separada.

8.6 Entregables

Entregables

- Informe técnico con capturas de pantalla del proceso completo.
- Tabla de vulnerabilidades y plan de hardening justificado.
- Archivo **.cap** con el handshake capturado.
- Declaración firmada de uso ético de las herramientas.
- Repositorio con scripts de automatización del proceso de auditoría.

8.7 Rúbrica de Evaluación

Cuadro 8.3: Rúbrica — Práctica P-C8 (10 puntos)

Criterio	Peso	Indicadores de logro
Presentación y defensa	20 %	Explica el flujo del ataque; propone contramedidas concretas; demuestra comprensión ética del proceso.
Informe escrito	20 %	Capturas completas; tabla de vulnerabilidades bien justificada; bibliografía correcta.
Calidad técnica	60 %	Handshake capturado correctamente; ataque de diccionario ejecutado; velocidad de ataque documentada; plan de hardening con ≥ 5 medidas justificadas.

Capítulo 9

P-C9: Diseño de Red Inalámbrica Empresarial

Unidad:	800-900 – Diseño y arquitectura empresarial	Código:	INGE-00100
Modalidad:	Presencial	Valoración:	10 puntos
Grupos:	Máx. 3 estudiantes	Entrega:	Según calendario

9.1 Marco Teórico

9.1.1 Proceso de diseño de redes inalámbricas

El diseño de una WLAN empresarial sigue el proceso **PPDIOO** (*Prepare, Plan, Design, Implement, Operate, Optimize*) de Cisco y debe considerar:

1. **Análisis de requerimientos:** número de usuarios concurrentes, tipos de dispositivos, aplicaciones críticas (VoIP, videoconferencia, IoT), cobertura requerida y presupuesto.
2. **Survey del sitio (*site survey*):** levantamiento físico para identificar obstáculos, fuentes de interferencia y zonas de cobertura requeridas.
3. **Selección de tecnología y equipos:** banda (2,4 / 5 / 6 GHz), estándar (802.11ac/ax), capacidad por AP y controladora WLAN.
4. **Planificación de canales y potencias:** diseño del patrón de frecuencias para minimizar la interferencia co-canal (CCI) e inter-canal (ACI).
5. **Seguridad:** segmentación en VLANs, AAA (802.1X + RADIUS), políticas de acceso por rol y detección de intrusiones (WIDS).

9.1.2 Herramientas de diseño

- **Cisco Packet Tracer:** simulador de redes con soporte para switches, routers, APs y clientes inalámbricos.
- **EKahau Site Survey:** herramienta profesional de *predictive* y *active site survey* (uso en versión de prueba o Ekahau HeatMapper gratuito).

9.2 Objetivos

Objetivos de Aprendizaje

Objetivo general

Diseñar, simular y documentar una red inalámbrica empresarial completa para un edificio de oficinas de tres plantas.

Objetivos específicos

- OE1.** Levantar los requerimientos de red del escenario propuesto mediante análisis de capacidad.
- OE2.** Seleccionar y ubicar los APs mediante planificación de celdas.
- OE3.** Configurar la red en Cisco Packet Tracer incluyendo VLANs, SSID por perfil de usuario y seguridad WPA2-Enterprise.
- OE4.** Verificar conectividad, segmentación y políticas de acceso mediante pruebas funcionales en el simulador.

9.3 Escenario

Empresa: “TeleSur S.A.”, empresa de servicios tecnológicos.

Edificio: 3 plantas, 40 × 20 m por planta.

Requerimientos:

- **Planta 1 — Atención al cliente:** 30 clientes concurrentes en red de invitados (acceso solo a Internet), 10 empleados en red corporativa.
- **Planta 2 — Oficinas:** 40 empleados con laptops + 20 teléfonos IP Wi-Fi (QoS WMM).
- **Planta 3 — Data Center y Servidores:** solo acceso autorizado, red de gestión separada.

Cuadro 9.1: VLANs y SSIDs propuestos para TeleSur S.A.

VLAN	Nombre	SSID	Seguridad	Perfil
10	Corporativa	TeleSur-Corp	WPA2-Enterprise	Empleados
20	Invitados	TeleSur-Guest	WPA2-Personal	Clientes
30	VoIP	TeleSur-Voice	WPA2-Enterprise	Teléfonos IP
40	Gestión	(no difundido)	WPA2-Enterprise	Admins

9.4 Procedimiento

9.4.1 Paso 1 — Cálculo de capacidad

Para cada planta calcule:

1. **Número de APs necesarios:** asuma que cada AP 802.11ax puede atender 30 dispositivos activos. Con la fórmula: $N_{AP} = \lceil N_{usuarios}/30 \rceil$.
2. **Radio de celda:** con una potencia de TX de 17 dBm y sensibilidad de cliente de -75 dBm, calcule el radio máximo de cobertura usando la Ecuación 7.1 y asuma un factor de penetración de pared de 3 dB por pared.
3. **Distribución de APs:** ubique los APs en el plano del edificio asegurando al menos -67 dBm de señal en toda el área de cobertura.

9.4.2 Paso 2 — Diseño en Cisco Packet Tracer

1. Cree la topología con: switch de distribución (capa 3), switches de acceso por planta y APs.
2. Configure las VLANs en el switch de distribución y los trunks hacia los switches de acceso.
3. Configure cada AP con los cuatro SSIDs mapeados a sus VLANs.
4. Configure el servidor DHCP con pools para cada VLAN.
5. Agregue un servidor RADIUS para autenticación 802.1X de las VLANs corporativas.

9.4.3 Paso 3 — Verificación

1. Pruebe conectividad desde un cliente de la VLAN Invitados: debe llegar a Internet pero **no** a la red Corporativa.
2. Pruebe conectividad desde un cliente Corporativo: debe acceder a servidores internos e Internet.
3. Verifique la segmentación con **ping** y captura de paquetes en Packet Tracer.
4. Documente la topología final con un diagrama de red lógico y físico.

9.5 Preguntas de Análisis

- P1. ¿Cuál es la diferencia entre WPA2-Personal y WPA2-Enterprise en cuanto a la gestión de credenciales y la escalabilidad?
- P2. ¿Qué es un *site survey* predictivo vs. activo? ¿Cuándo se recomienda cada uno?
- P3. Explique cómo funciona la autenticación 802.1X con RADIUS en una red Wi-Fi empresarial.

- P4.** ¿Cómo se aplica QoS (WMM) para priorizar el tráfico VoIP sobre el tráfico de datos en una WLAN?
- P5.** ¿Qué es un WIDS (*Wireless Intrusion Detection System*) y cómo integraría uno en el diseño de TeleSur S.A.?

9.6 Entregables

Entregables

1. Informe técnico con análisis de requerimientos y justificación de decisiones de diseño.
2. Diagrama de red lógico (VLANs, SSIDs, IPs) y físico (ubicación de APs por planta).
3. Archivo **.pkt** de Cisco Packet Tracer con la topología completa y configuraciones.
4. Capturas de pantalla de las pruebas de verificación (conectividad y segmentación).
5. Presupuesto estimado de implementación (equipos + licencias).

9.7 Rúbrica de Evaluación

Cuadro 9.2: Rúbrica — Práctica P-C9 (10 puntos)

Criterio	Peso	Indicadores de logro
Presentación y defensa	20 %	Expone la arquitectura de red; justifica las decisiones de diseño; demuestra la simulación en vivo.
Informe escrito	20 %	Diagramas claros y completos; análisis de requerimientos detallado; bibliografía correcta.
Calidad técnica	60 %	Topología funcional en Packet Tracer; VLANs correctamente configuradas; segmentación verificada; RADIUS operativo; cálculo de capacidad justificado.

Referencias Bibliográficas

- [1] K. Ok, V. Coskun, and B. Ozdenizci, *Near Field Communication: From Theory to Practice*. ISIK University, 2012.
- [2] W. Dargie and C. Poellabauer, *Fundamentals of Wireless Sensor Networks: Theory and Practice*. Chichester: John Wiley & Sons, 2010.
- [3] W. Stallings, *Wireless Communications & Networks*, 2nd ed. Pearson Education India, 2009.
- [4] J. H. Schiller, *Mobile Communications*, 2nd ed. Pearson Education, 2003.
- [5] W. Osterhage, *Wireless Network Security*. CRC Press, 2018.
- [6] Espressif Systems, “ESP32 Technical Reference Manual,” Rev. 5.1, 2023. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf
- [7] M. Balboa, “esp-rc522 — ESP-IDF RC522 driver,” GitHub, 2023. [Online]. Available: <https://github.com/miguelbalboa/rfid>
- [8] NXP Semiconductors, “MFRC522 Standard performance MIFARE and NTAG Frontend,” Product Data Sheet Rev. 3.9, 2016. [Online]. Available: <https://www.nxp.com/docs/en/data-sheet/MFRC522.pdf>
- [9] Universidad de Cuenca, *Libro de Marca UCUENCA*. Cuenca: Universidad de Cuenca, 2021.